A Survey of AIOps in the Era of Large Language Models

LINGZHE ZHANG, Peking University, China
TONG JIA*, Peking University, China
MENGXI JIA, Peking University, China
YIFAN WU, Peking University, China
AIWEI LIU, Tsinghua University, China
YONG YANG, Peking University, China
ZHONGHAI WU, Peking University, China
XUMING HU, The Hong Kong University of Science and Technology (Guangzhou), China
PHILIP S. YU, University of Illinois Chicago, United States
YING LI*, Peking University, China

arXiv:2507.12472v1 [cs.SE] 23 Jun 2025

As large language models (LLMs) grow increasingly sophisticated and pervasive, their application to various Artificial Intelligence for IT Operations (AIOps) tasks has garnered significant attention. However, a comprehensive understanding of the impact, potential, and limitations of LLMs in AIOps remains in its infancy. To address this gap, we conducted a detailed survey of LLM4AIOps, focusing on how LLMs can optimize processes and improve outcomes in this domain. We analyzed 183 research papers published between January 2020 and December 2024 to answer four key research questions (RQs). In RQ1, we examine the diverse failure data sources utilized, including advanced LLM-based processing techniques for legacy data and the incorporation of new data sources enabled by LLMs. RQ2 explores the evolution of AIOps tasks, highlighting the emergence of novel tasks and the publication trends across these tasks. RQ3 investigates the various LLM-based methods applied to address AIOps challenges. Finally, RQ4 reviews evaluation methodologies tailored to assess LLM-integrated AIOps approaches. Based on our findings, we discuss the state-of-the-art advancements and trends, identify gaps in existing research, and propose promising directions for future exploration.

CCS Concepts: • **Software and its engineering → Maintaining software**; • **Computing methodologies → Artificial intelligence**; • **General and reference → Surveys and overviews**.

Additional Key Words and Phrases: Large Language Model, AIOps, Metrics, Logs, Time Series, Incident Report, Failure Perception, Anomaly Detection, Root Cause Analysis, Assisted Remediation

*Corresponding author.
Authors' addresses: Lingzhe Zhang, Peking University, Beijing, China, zhang.lingzhe@stu.pku.edu.cn; Tong Jia*, Peking University, Beijing, China, jia.tong@pku.edu.cn; Mengxi Jia, Peking University, Beijing, China, mxjia@pku.edu.cn; Yifan Wu, Peking University, Beijing, China, yifanwu@pku.edu.cn; Aiwei Liu, Tsinghua University, Beijing, China, liuaw20@mails.tsinghua.edu.cn; Yong Yang, Peking University, Beijing, China, yang.yong@pku.edu.cn; Zhonghai Wu, Peking University, Beijing, China, wuzh@pku.edu.cn; Xuming Hu, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China, xuminghu@hkust-gz.edu.cn; Philip S. Yu, University of Illinois Chicago, Chicago, United States, psyu@uic.edu; Ying Li*, Peking University, Beijing, China, li.ying@pku.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
0004-5411/2025/8-ART111 $15.00
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Lingzhe Zhang, Tong Jia*, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, and Ying Li*. 2025. A Survey of AIOps in the Era of Large Language Models. *J. ACM* 37, 4, Article 111 (August 2025), 35 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Modern software systems are growing increasingly complex and typically serve massive user bases numbering in the billions [55, 160]. Even minor software glitches can lead to substantial financial and reputational losses due to service interruptions or degraded performance [26]. As a result, large-scale distributed systems must ensure continuous 24/7 operation, with high demands for availability and reliability. However, the sheer scale and intricate logic of these systems make failures both inevitable and difficult to detect, localize, and diagnose. Once faults occur, analysis and recovery become even more challenging. Consequently, effective AIOps—encompassing rapid failure detection, efficient diagnosis, accurate root cause identification, and timely remediation—has become essential for maintaining system reliability and availability.

Scope of AIOps in this paper. Artificial Intelligence for IT Operations (AIOps) was first introduced by Gartner in 2016 [104]. AIOps refers to the use of machine learning (ML) and deep learning (DL) techniques to process vast amounts of data generated by operational tools and infrastructure, enabling the real-time detection, diagnosis, and resolution of system issues. This greatly enhances the automation and intelligence of IT operations. Most existing literature adopts this scope of AIOps, focusing on its application to operational reliability and failure management [17, 97, 108, 121, 134]. However, some works have extended the definition of AIOps to include topics such as Ops for AI and network security [21, 96]. These directions, while valuable, are beyond the scope of this paper. We focus specifically on the role of AIOps in addressing runtime software failures in large-scale distributed systems.

1.1 Why are LLMs Beneficial for AIOps?

With the rapid advancement of Artificial Intelligence (AI), traditional AIOps approaches based on machine learning (ML) and deep learning (DL) have played a pivotal role in automated software failure management [17, 21, 97, 108, 134]. However, in practical applications, these methods still face several challenges:

- **Need for complex feature extraction engineering.** These methods typically require extensive preprocessing and feature engineering to extract useful information from raw data. They have limited capabilities in understanding and processing data, especially in handling unstructured data such as logs and traces, which appearing relatively weak.
- **Lack of cross-platform generality.** Traditional models are often tuned and trained specifically for a particular software system. Once a different software system is adopted or even minor changes are made to the original system, the performance of the model significantly deteriorates, even when performing the same task.
- **Lack of cross-task flexibility.** Due to the singularity of model knowledge and outputs, models can only perform one task at a time. For example, in Root Cause Analysis (RCA) tasks, some work is aimed at identifying the cause of the problem [73, 122, 147], while others are focused on identifying the software components involved [46, 92, 130]. In real-world scenarios, multiple models must run simultaneously to complete the entire RCA task.
- **Limited model adaptability.** With changes in the software system, deep learning-based methods typically require frequent model training and updates to adapt to new data and environments. While there are many online learning methods [1, 37, 69, 82] available to

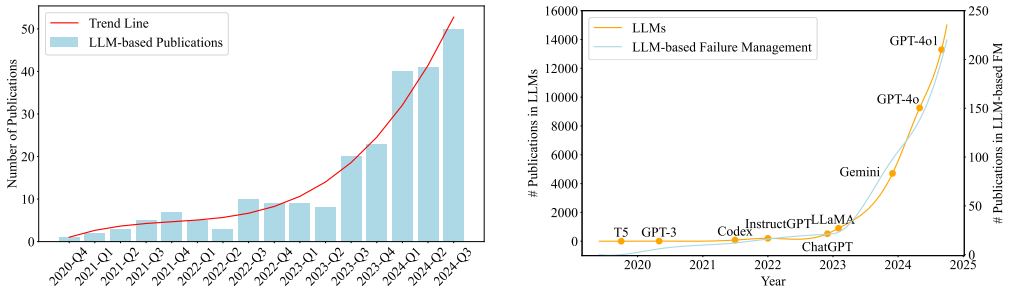
address this issue, this process not only consumes time and effort but also can result in delayed responses from the model when handling sudden events.

- **Restricted levels of automation.** Current deep learning-based methods exhibit relatively limited capabilities in terms of automated operations and intelligent decision-making. While some degree of automation is achievable, significant manual intervention and configuration are still required. Particularly in the case of Auto Remediation, current efforts are mainly stopped at Incident Triage [148] or Solution Recommendation [131, 175].

Large Language Models (LLMs), pre-trained on natural language understanding tasks, offer a promising avenue for addressing these limitations. (1) Due to their robust natural language processing capabilities, LLMs can efficiently handle and comprehend unstructured data, often eliminating the need for prior feature extraction. (2) Trained on vast amounts of cross-platform data, LLMs possess a strong degree of generality and logical reasoning abilities. (3) Outputting natural language, LLMs offer great flexibility, enabling them to simultaneously perform multiple AIOps tasks, such as identifying the cause of a problem and the involved software components. (4) Leveraging their pre-training, LLMs exhibit powerful adaptive capabilities and can incorporate continuously updated external knowledge using methods like Retrieval-Augmented Generation (RAG), often without requiring retraining. (5) With robust script generation capabilities and the ability to automatically invoke external tools, LLMs can achieve higher levels of automation.

1.2 Why a Survey of AIOps in the Era of LLMs?

Due to the advantages of large language models, an increasing number of efforts in the field of AIOps are now leveraging LLMs, as shown in Figure 1(a). Furthermore, the rapid development of LLMs has spurred a thriving research landscape in LLM-based AIOps, as illustrated in Figure 1(b). Notably, with the advent of ChatGPT, research on LLM-based AIOps has experienced a significant surge in interest. In summary, as time progresses, the number of works utilizing LLMs continues to grow, and the pace of growth is expected to accelerate in the near future. Therefore, a comprehensive survey analyzing the evolution of AIOps in the era of large models is needed.



(a) Number of LLM-based publications in the field of AIOps

(b) Number of publications in the field of LLM-based AIOps and LLMs (the data for "# Publications in LLMs" is extended based on [171])

Fig. 1. Analysis of Publication Trends in LLM-Based AIOps

In fact, in recent years, numerous literature reviews have summarized research on AIOps. As shown in Table 1, these works are either based on traditional machine learning or deep learning algorithms and do not use LLM-based approaches [17, 97, 108, 134], or they do not provide a systematic summary of all tasks involved in the full process of AIOps [121].

Table 1. Comparison of existing AIOps surveys

Reference	Year	Scope of AIOps Tasks	LLM-based
Paolop et al. [97]	2021	Failure Perception; Root Cause Analysis; Remediation	
Qian et al. [17]	2023	Incident Detection; Failure Prediction; Root Cause Analysis; Automated Actions	
Youcef et al. [108]	2024	Incident Reporting; Incident Triage; Incident Diagnosis; Incident Mitigation	
Wei et al. [134]	2024	Anomaly Detection	
Jing et al. [121]	2024	Time-series Forecasting; Anomaly Detection	✓
Our work	-	Data Preprocessing; Failure Perception; Root Cause Analysis; Auto Remediation	✓

In summary, comprehensive studies exploring AIOps in the era of LLMs are lacking. In this study, we present the first comprehensive survey that covers the entire process of AIOps in the context of large language models. This survey aims to provide researchers with an in-depth understanding of LLM-based approaches for AIOps, facilitating comparisons and contrasts among different methods. It also guides users interested in applying LLM-based AIOps methods by helping them choose suitable algorithms for different application scenarios.

1.3 Research questions

With the rapid emergence of numerous LLM-based AIOps solutions—and the strong momentum indicating even greater growth—a comprehensive survey of AIOps in the era of LLMs has become both timely and necessary. The main goal of this survey is to systematically analyze the emerging trends and remaining challenges in LLM-driven AIOps research. By doing so, we aim to offer practical guidance for industrial applications and provide valuable insights to inform future academic research.

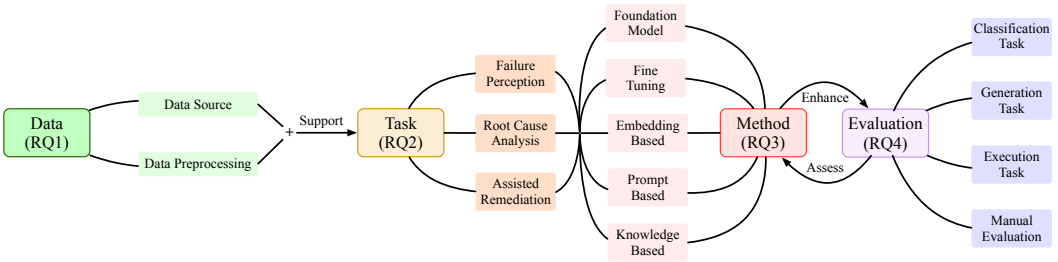


Fig. 2. Taxonomy of Research Questions

To achieve this, we organize our survey around the following research questions, each targeting a critical aspect of how large language models are reshaping AIOps:

- **RQ1:** How has the advent of LLMs transformed the sources and preprocessing methods of data in AIOps?
- **RQ2:** How have the tasks of AIOps evolved with the advent of LLMs?

- **RQ3:** What methods leveraging LLMs are employed in AIOps?
- **RQ4:** How have evaluation methodologies in AIOps adapted to the integration of LLMs?

The taxonomy of these research questions is illustrated in Figure 2. Specifically, RQ1 focuses on data sources and preprocessing techniques, which serve as the foundation for the various AIOps tasks examined in RQ2. These tasks are supported by the LLM-based methods reviewed in RQ3. Finally, RQ4 discusses how these methods are assessed through different evaluation strategies. Notably, LLM-based methods may also drive improvements in evaluation itself, forming a feedback loop that continually enhances AIOps performance.

1.4 Structure of the paper

This reminder of this survey is structured as follows: In Section 2, we outline the methods used in the systematic review and explain the search strategy. Sections 3 through 6 present the findings relevant to each of the four research questions. Section 7 discusses ongoing challenges and potential future research directions in LLM-based AIOps. Finally, the survey concludes in Section 8.

2 SYSTEMATIC REVIEW PROCESS

To conduct a systematic review, we first establish a research protocol that includes the following steps: defining the search strategy and scope, setting inclusion and exclusion criteria, and presenting an overview of the selected publications [57, 61]. The purpose of this research protocol is to ensure the study is conducted in a clear, rigorous, and reproducible manner.

2.1 Search strategy

To manage the vast body of literature efficiently, we limit our search to five databases[1]. Scopus and Web of Science are included due to their comprehensive indexing of peer-reviewed journals and their widespread use in systematic reviews. Additionally, we select IEEE Xplore and the ACM Digital Library, as they are frequently cited in related literature surveys and cover a broad range of applied research fields. Finally, we include arXiv, given its prominence as the leading pre-print platform in the field of computer science.

Based on our preliminary investigation, we identify relevant keywords for searching literature on LLM-based AIOps and limited our search to papers matching the search string shown in Figure 3 and focus on works published after 2020. To ensure a comprehensive collection of papers, we include broader keywords during the search, such as *IT Operations* and *Pre-trained Model*. Subsequently, we manually filter out works that fall outside the scope of this survey using exclusion criteria.

ALL_FIELD ('Large Language Model' **OR** 'Pre-trained Model') **AND**
TITLE ('Failure Management' **OR** 'Incident Management' **OR** 'Failure Detection' **OR** 'Anomaly Detection' **OR** 'Failure Prediction' **OR** 'Root Cause' **OR** 'Failure Localization' **OR** 'Failure Diagnosis' **OR** 'Anomaly Diagnosis' **OR** 'Remediation' **OR** 'Log Analysis' **OR** 'Log Understanding' **OR** 'Log Parsing' **OR** 'Alert' **OR** 'Self-Healing' **OR** 'Failure Mitigation' **OR** 'Recommended Resolutions Generation' **OR** 'Time Series Forecast' **OR** 'IT Ops' **OR** 'Service Composition' **OR** 'Deadlock Prevention' **OR** 'Deadlock Prediction' **OR** 'Misconfiguration Detection')

Fig. 3. Search Strategy Utilized to Identify Studies on AIOps

[1]https://www.scopus.com/, https://www.webofscience.com/, https://ieeexplore.ieee.org/, https://dl.acm.org/, https://arxiv.org/

2.2 Selection criteria

The criteria for selecting papers were as follows. Papers are included if they were deemed relevant to at least one of the proposed research questions. Specifically, a paper is selected if it addresses any of the following: introducing new data sources (RQ1), proposing novel LLM-based methods (RQ2), discussing emerging trends in AIOps tasks (RQ3), or introducing new evaluation metrics or datasets (RQ4). To ensure relevance, papers have to satisfy at least one of the following inclusion criteria:

- **IC1:** The paper introduces new data sources for AIOps.
- **IC2:** The paper presents novel LLM-based methods for AIOps.
- **IC3:** The paper discusses new changes in AIOps tasks in the LLM era.
- **IC4:** The paper proposes new evaluation metrics or datasets tailored for LLM-based AIOps.

Judgments are primarily made based on titles and abstracts. In cases of uncertainty, the full paper is reviewed. To exclude irrelevant works, we apply a series of exclusion criteria to the included papers. Papers are removed if they meet any of the following conditions:

- **EC1:** The study uses a model with a parameter size smaller than 1 billion.
- **EC2:** The study is unrelated to software systems.
- **EC3:** The study is outside the scope of AIOps.
- **EC4:** The study has been retracted or the full text is unavailable.
- **EC5:** The study only presents conceptual ideas without experimental results.

Each record is reviewed by a primary reviewer, with non-trivial cases discussed and resolved with at least one secondary reviewer. The final set of selected papers is validated by all co-authors. The exclusion criteria are established as follows: Papers have to demonstrate the utilization of large language models. Studies employing smaller models, particularly BERT-based ones, are excluded, as BERT is not widely considered an LLM. Additionally, papers are required to focus on software systems, resulting in the exclusion of studies from other domains, particularly those related to robotics or mechanical devices. Given the broad scope of our search strategy, which included keywords such as IT Operations, we manually filter out papers outside the domain of AIOps. Finally, EC4 and EC5 are used to eliminate low-quality studies, such as retracted papers or those lacking experimental results.

2.3 Overview of selected publications

We then present an overview of the workflow for selecting the papers and summarize the corresponding results. As shown in Figure 4, we initially retrieve 761 papers from five databases. After removing duplicates, 614 papers remain. During the screening phase, we exclude 333 papers based on their titles and abstracts using criteria EC1-EC4. This step primarily filter out papers unrelated to AIOps. The remaining 395 papers are downloaded for detailed reading, where we find that 222 papers do not align with the scope of our study. These papers predominantly utilize small-scale models that do not meet our requirements. Ultimately, 163 new studies satisfy our predefined inclusion criteria and are selected for further analysis in this systematic review.

3 RQ1: TRANSFORMATIONS IN DATA WITH LLM INTEGRATION

In this section, we examine the data sources employed in various studies to explore how data has transformed in the era of LLMs. First, we analyze the new techniques applied to process traditional data sources. Then, we discuss the novel data sources introduced for AIOps in the LLM era.

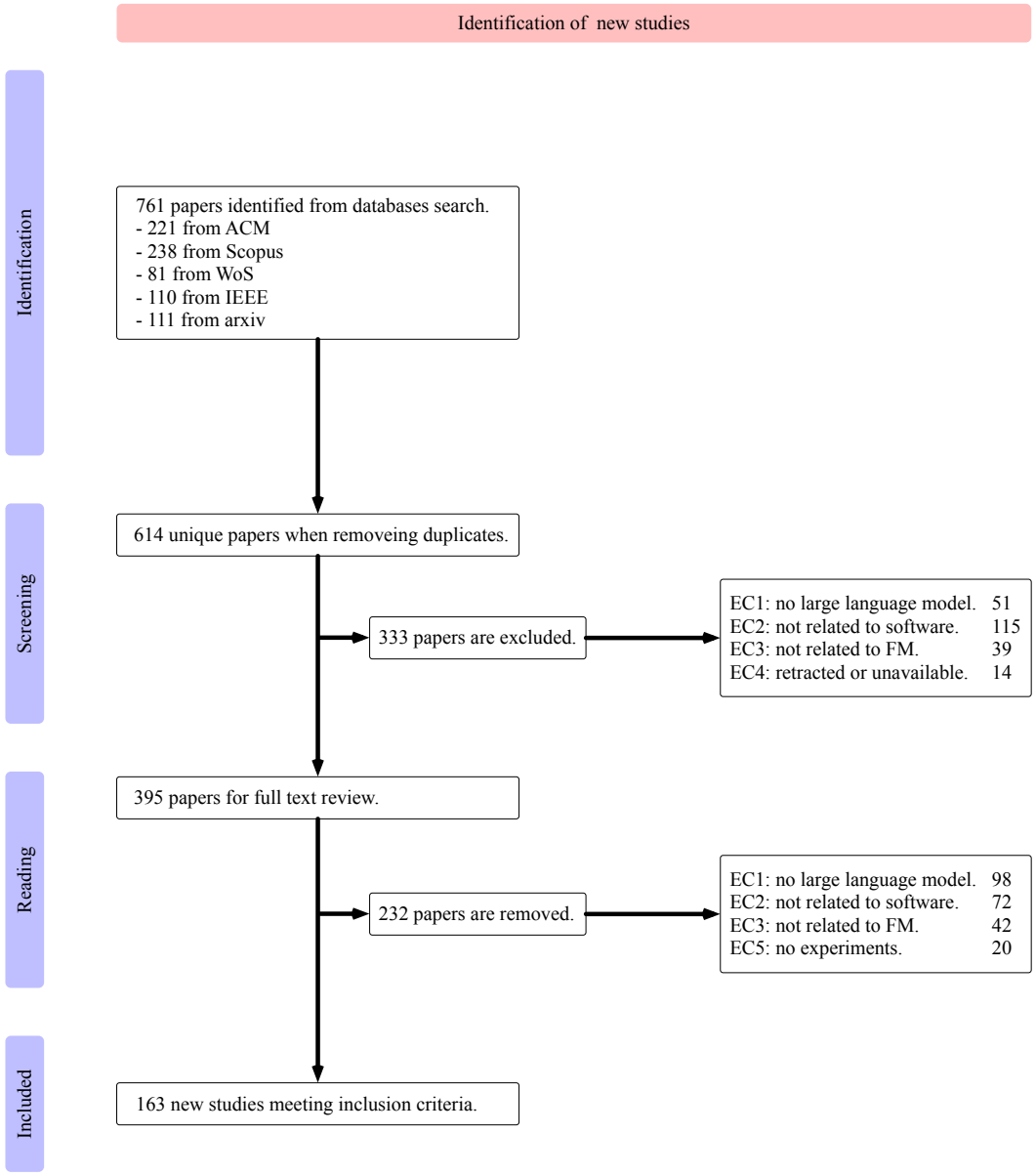


Fig. 4. Overview of Paper Selection Procedure

3.1 Advancing Preprocessing Techniques for Traditional Data Sources

To begin, we review the data sources traditionally employed in ML-based and DL-based AIOps [68, 73, 122, 147, 149, 150, 152, 154, 157, 176, 178]. These studies primarily rely on runtime data generated by the system, which encapsulates rich intrinsic and extrinsic information about the operation of software systems. This data can be categorized into three main types: **metrics**, **logs**, and **traces**.

- Metrics are quantitative measurements collected from various components of the IT infrastructure, such as CPU usage, memory utilization, disk I/O, network latency, and throughput. These metrics provide real-time performance insights and are critical for monitoring the system's health and performance.
- Logs are detailed records of events occurring within the system. They encompass error messages, transaction records, user activities, and system operations. Logs are essential for diagnosing issues, analyzing system behavior, and tracking historical changes over time.
- Traces capture the sequence of operations or transactions that a request undergoes in a distributed system. They offer a high-level perspective on the interactions between services, aiding in the identification of performance bottlenecks, service dependencies, and root causes of issues within microservices architectures.

In metrics data, a significant challenge stems from the high dimensionality and granularity of data—potentially thousands of indicators generating data points every second. During transmission or storage, data loss is common, and such missing values do not necessarily indicate anomalies in the system. Therefore, imputing missing data is critical, as it can significantly improve the performance of downstream tasks. While many LLM-based metrics methods address imputation as part of broader objectives such as anomaly detection, several studies specifically target the imputation problem. For example, GatGPT [15] employs a graph attention network to pre-train a large language model tailored for metrics imputation. Jacobsen et al. [47] apply parameter-efficient fine-tuning techniques to adapt LLMs for this task. CRILM [39] takes a different approach by using LLMs to generate contextually informed descriptors for missing values, rather than relying on numerical interpolation.

In trace data, missing or incomplete traces may result from failures in the monitoring infrastructure. To address this, trace generation has emerged as a promising new direction. Although LLM-based research in this area is still limited, Kim et al. [60] propose a pioneering approach that fine-tunes a large language model to synthesize realistic workload traces, particularly in the form of microservice call graphs.

Among the various data sources in AIOps, log data has received the most attention in preprocessing research. This is largely due to its unstructured nature, high information density, and critical role in capturing system behaviors. Within this area, log parsing has emerged as the most actively studied task. A significant number of studies leverage LLMs to convert raw log messages into structured representations, which serve as essential inputs for downstream tasks such as failure detection and root cause analysis.

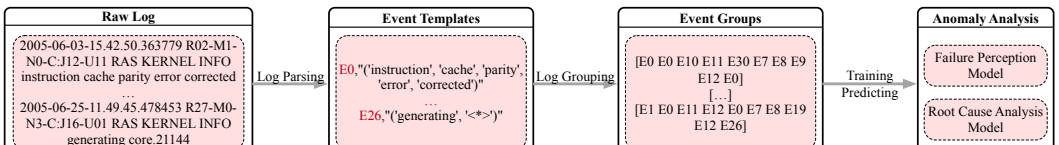


Fig. 5. Log-based Failure Perception and Root Cause Analysis: The Common Workflow

As shown in Figure 5, raw logs consist of semi-structured text encompassing various fields like timestamps and severity levels. For the benefit of downstream tasks, log parsing is employed to transform each log message into a distinct event template, which includes a constant part paired with variable parameters. For example, the log template "E0,('instruction', 'cache', 'parity', 'error', 'corrected')" can be extracted from the log message "2005-06-03-15.42.50.363779 R02-M1-N0-C:J12-U11 RAS KERNEL INFO instruction cache parity error corrected" in Figure 5. After being parsed

into event templates, log data can be organized into sequence groups using session, sliding, or fixed windows. Following this, failure perception and root cause analysis are performed on each event group to determine if a failure exists, and if so, to conduct the corresponding root cause analysis.

A common issue in traditional log parsing methods is their lack of generalizability. These methods often rely on manually designed rules or are trained on limited datasets. As a result, their effectiveness significantly decreases when applied to different software systems or when there are changes in log generation rules. The emergence of powerful LLMs, which possess extensive pre-trained knowledge related to code and logging, offers a promising avenue for log parsing. However, the lack of specialized log parsing capabilities currently hinders the accuracy of LLMs in this task. Additionally, the inherent inconsistencies in their responses and the substantial computational overhead prevent the practical adoption of LLM-based log parsing. Consequently, many studies have explored the effectiveness of LLMs for log parsing and have proposed methods to effectively leverage LLMs in this domain.

Empirical Study. Priyanka et al. [94] conducted a study on zero-shot log parsing using ChatGPT. Their findings indicated that the current version of ChatGPT has limited performance in zero-shot log processing, with issues of response inconsistency and scalability. Le et al. [66] evaluated ChatGPT's ability to perform log parsing through two research questions: the effectiveness of ChatGPT in parsing logs and its performance with different prompting methods. Their results showed that ChatGPT can achieve promising outcomes in log parsing with appropriate prompts, especially with a few-shot approach. These empirical studies demonstrate that while LLMs have the potential for log parsing, they require effective methods to guide them.

Prompt-based Approach. Some studies have adopted prompt-based methods to guide LLMs for effective log parsing. LILAC [51] proposes a practical log parsing framework using LLMs with an adaptive parsing cache. LILAC leverages the in-context learning (ICL) capabilities of LLMs by executing a hierarchical candidate sampling algorithm and selecting high-quality demonstrations. It uses an adaptive parsing cache to store and optimize LLM-generated templates, helping mitigate the inefficiency of LLMs by quickly retrieving previously processed log templates. LLMParser [86] employs in-context learning and few-shot tuning methods. This approach is tested on four LLMs: Flan-T5-small, Flan-T5-base, LLaMA-7B, and ChatGLM-6B, across 16 open-source systems, and find that smaller LLMs might be more effective for log parsing tasks than more complex ones. Lemur [33] introduces a novel sampling method inspired by information entropy, effectively clustering typical logs and using the Chain of Thoughts (CoT) method with LLMs to distinguish parameters from invariant tokens. DivLog [140] combines Retrieval-Augmented Generation (RAG) and ICL methods for log parsing, sampling a diverse set of offline logs as candidate logs and selecting five suitable template candidates for each target log during the parsing process. Sun et al. [124] develop a cloud-native log management platform providing log collection, transmission, storage, and system management features, utilizing GPT-3.5 for few-shot log parsing.

Fine-tuning Approach. Other studies have adopted fine-tuning approaches to train pre-trained models specifically for log parsing. OWL [32] employs supervised fine-tuning and mixture adapter tuning techniques to develop a set of large language models for knowledge querying and log parsing, leveraging the LLaMA model and their OWL-Instruct dataset. LogLM [78] utilizes instruction-based fine-tuning on LLaMA2-7B with seven Loghub datasets [41], enabling the base model to acquire log parsing capabilities. Mehrabi et al. [83] fine-tune the Mistral-7B-Instruct LLM and demonstrate that a fine-tuned, compact LLM can achieve comparable or even superior results to those of large-scale LLMs in log parsing.

3.2 Emerging Data Sources in AIOps

The aforementioned traditional data sources are all auto-generated by system, so in this survey we categorize them as system-generated data. However, with the emergence of LLMs, many approaches have started to incorporate information created by humans to provide auxiliary knowledge for managing software system failures [2, 8, 9, 29, 32, 36, 50, 54, 109, 119, 151, 170, 179]. These human-generated data types include **Software Information**, **Question & Answer (QA)**, and **Incident Reports**.

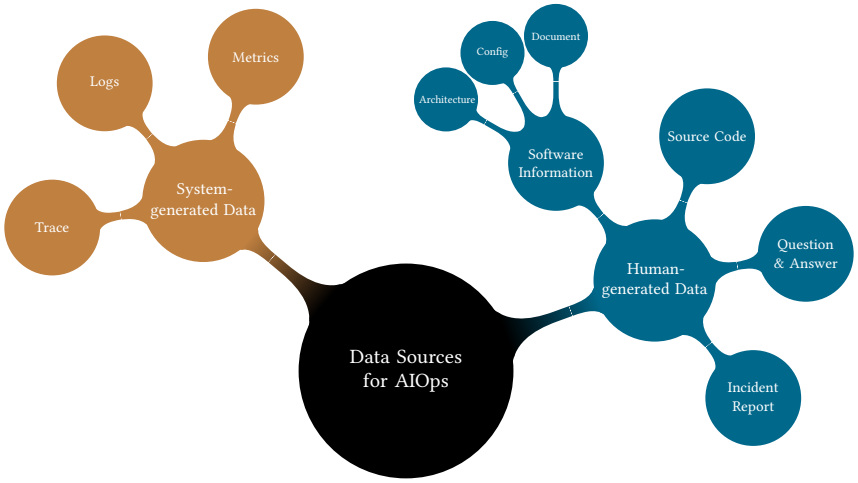


Fig. 6. Data Source for AIOps in the Era of LLM

Software Information. Software information is generated during the software development process and includes details such as *software architecture*, *configurations* and *documentation*. This type of information provides valuable knowledge about the development process, helping AIOps approaches to perform better. By leveraging software information, failure mangement methods can gain insights into the system's design and functionality [8, 29, 64, 71, 87, 91, 116, 119, 133, 135, 138, 177, 179, 180], which can be crucial for diagnosing issues and implementing effective solutions. For example, Goel et al. [29] experimentally demonstrate that incorporating dependent-service descriptions significantly improves the performance of root cause analysis. Moreover, providing service architecture and functionality information helps LLMs more effectively identify the SLO (Service Level Objective) categories of monitors.

Source Code. Source code contains the foundational instructions and logic that define the behavior of software systems. It serves as a critical resource for AIOps by providing deep insights into the system's structure, functionality, and potential areas of vulnerability. In the era of LLMs, source code analysis has been significantly enhanced through automated code comprehension, bug detection, and the generation of potential fixes. For instance, LLMs can analyze source code to identify patterns indicative of bugs or inefficiencies, enabling proactive issue identification. Furthermore, the ability of LLMs to generate code snippets or modifications directly supports tasks such as automated remediation and optimization, bridging the gap between fault diagnosis and corrective action. By leveraging source code, AIOps methods gain a robust foundation for understanding and addressing complex software failures [20, 59, 95, 98, 133]. For example, RCAgent [133] enhances root cause diagnosis for Flink runtime anomalies by jointly leveraging logs and source code. It first uses logs to locate suspicious components and then retrieves the corresponding code snippets.

These snippets are subsequently analyzed by an LLM, enabling a deeper semantic understanding of the logs and improving the accuracy of root cause identification.

Question & Answer (QA). QA data consists of question-and-answer pairs related to operational or developmental knowledge. In the context of LLMs, a rich repository of QA data can serve as a knowledge base, functioning similarly to a search engine, to provide valuable assistance to operations personnel. Additionally, like software information, QA data can offer auxiliary knowledge that enhances AIOps strategies, facilitating quicker and more accurate issue resolution [8, 32, 79, 166]. For example, OWL [32] fine-tunes LLaMA2-13B specifically for IT operations-related question answering tasks, and achieves superior performance compared to existing state-of-the-art models.

Incident Reports. Incident reports are often written by software users. When an incident is created, the author specifies a title for the incident and describes relevant details such as error messages, anomalous behavior, and other information that could help with resolution. Before the emergence of LLMs, these incident reports were submitted to on-call engineers (OCEs) for diagnosis. However, due to the powerful natural language processing capabilities of LLMs, many approaches now automatically analyze these incident reports, diagnose faults, and even suggest mitigation steps [2, 29, 36, 50, 54, 59, 62, 109, 151, 170]. This automation enhances the efficiency of AIOps by quickly identifying and addressing issues based on the detailed information provided in the reports.

In summary, the advent of large language models has significantly expanded the data sources used for AIOps. In addition to traditional system-generated data, a substantial amount of human-generated data with semantic richness but less structured form has been leveraged. These human-generated data types provide valuable context and knowledge that were previously underutilized. By incorporating this new category of data, LLMs enable more comprehensive AIOps approaches, improving the system's ability to detect, diagnose, and remediate failures more effectively.

4 RQ2: EVOLVING TASKS IN AIOPS WITH LLMS

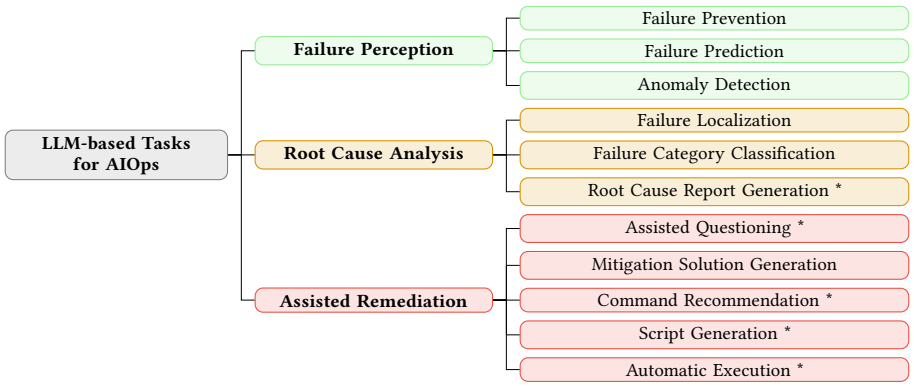


Fig. 7. AIOps Tasks in the Era of LLM (Tasks marked with * are new tasks that have emerged in the era of LLMs)

In this section, we present a comprehensive overview of how AIOps tasks have evolved in the era of large language models. We examine the full spectrum of tasks, encompassing both traditional and newly emerging ones, to highlight the transformative role LLMs play in this domain.

AIOps is a multifaceted process comprising several interconnected subtasks. As depicted in Figure 7, the workflow is divided into three primary stages: failure perception, root cause analysis, and assisted remediation. These stages are sequentially linked as follows:

Failure Perception: This stage focuses on detecting whether anomalies have occurred in the system.

Root Cause Analysis: Once an anomaly is detected, this stage identifies the location and nature of the issue through automated analysis.

Assisted Remediation: Building on the findings of root cause analysis, this stage employs appropriate methods to assist human operators in mitigating the problem effectively.

With the integration of large language models, numerous new subtasks have emerged within these tasks, while many traditional subtasks have undergone significant transformations.

4.1 Failure Perception

Failure perception is the foundational step in AIOps, playing a crucial role in the early detection of potential issues to enable proactive measures that prevent failures [101]. This process encompasses several subtasks, including failure prevention, failure prediction, and anomaly detection—all of which existed prior to the advent of large language models. These tasks primarily rely on logs and metrics as data sources, though a few recent studies have incorporated software-related information, particularly configuration data [87, 91, 135].

Failure Prevention. This task involves techniques such as software defect prediction and fault injection, which aim to assist in building robust systems and processes to minimize the likelihood of failures [97]. By identifying potential weaknesses and simulating failure scenarios, these methods help developers design systems with enhanced reliability and fault tolerance. However, in the era of large language models, there has been little work directly addressing failure prevention. The only related study, FAIL [7], uses LLMs to analyze news articles to preemptively address dependency issues, thereby contributing to failure prevention. This approach differs from traditional failure prevention tasks in its methodology and focus.

Failure Prediction. This task focuses on forecasting potential system failures before they occur by analyzing historical data and identifying patterns that typically precede such failures. This proactive approach alerts maintenance teams to emerging issues, enabling them to implement early remediation measures. However, LLM-based approaches in this domain remain limited. This scarcity arises from the inherent challenge that many failures occur without clear precursors, resulting in methodologies that are either restricted to addressing a narrow range of anomalies or exhibit high rates of missed detections (false negatives). Only a few studies have attempted to leverage LLMs to enhance the effectiveness of failure prediction models [4, 5, 139, 143], but the tasks they address in failure prediction are constrained in scope.

Anomaly Detection. Due to the inherent limitations of failure prediction, anomaly detection has emerged as a primary focus within failure perception. Its objective is to identify abnormal behaviors or patterns that deviate from the system's normal operations, serving as early indicators of potential issues or failures. This subtask has consistently been the most prominent area of research in failure perception, both before and after the advent of LLMs. With the integration of LLMs, recent efforts have shifted toward enhancing model generalization [11, 19, 25, 28, 35, 52, 58, 67, 76, 78, 84, 87, 107, 126, 142, 146] (e.g., developing or fine-tuning foundation models for time series and logs), leveraging large models to improve the performance of smaller ones [27, 48, 65, 115, 127, 153, 161, 162] (e.g., using LLMs to generate embeddings for logs), and circumventing model training entirely [6, 10, 13, 18, 23, 40, 42, 43, 52, 53, 75, 80, 94, 100, 105, 110, 117, 123, 125, 141, 145, 156, 165, 169, 173] (e.g., directly using prompts to predict the next metric or log entry).

4.2 Root Cause Analysis

Once anomalies are detected in a software system, it becomes crucial to identify the affected component and the specific nature of the anomaly [120]. Accurate root cause analysis is vital in

assisting operations personnel during the repair process. Based on the specific task at hand, these approaches can be categorized into failure localization, failure category classification, and root cause report generation. While failure localization and failure category classification are traditional tasks, root cause report generation is a new task that has emerged with the integration of LLMs. In the LLM era, this task is often addressed using multimodal data within a single workflow—primarily traces, metrics, and logs. A growing body of work, particularly from Microsoft, focuses on analyzing incident reports in this context [2, 29, 36, 50, 54, 59, 62, 109, 151, 170]. Additionally, software information and source code are frequently used to complement and enhance the analysis of these primary data sources.

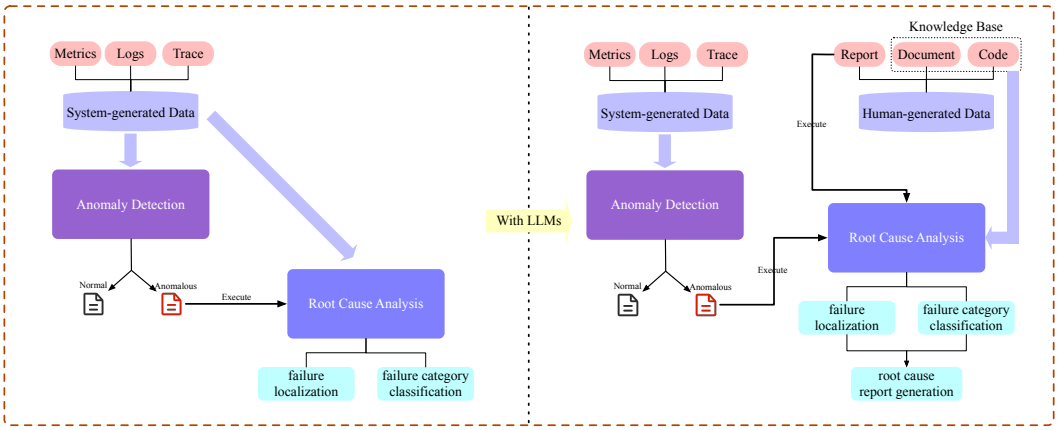


Fig. 8. Evolution of Root Cause Analysis with the Rise of LLMs

As illustrated in Figure 8, before the rise of large language models, these tasks typically relied on system-generated data and utilized automated failure perception methods to identify anomalies, which was then used for failure localization and failure category classification. However, with the advent of large language models, the starting point for root cause analysis has shifted from automated failure perception to user-generated data, particularly incident reports which has introduced the ability to analyze natural language. Additionally, root cause analysis now incorporates other human-generated data, such as documentation and code, as supplementary knowledge sources to enhance the analysis. Furthermore, based on the natural language understanding and generation capabilities of LLMs, it is now possible to bypass the processes of failure localization and failure category classification, and directly generate root cause reports.

Failure Localization. This task aims to identify the specific component or machine where an anomaly occurred, often using methods such as causal discovery. In microservices environments, it helps pinpoint the exact service or machine experiencing the issue. Additionally, it may involve identifying the specific log or metric entry that signals the onset of the anomaly. In the era of LLMs, many studies have focused on leveraging these models to locate erroneous configurations [87, 116, 135] and identify failure nodes in microservices systems [38, 109, 113, 114, 118, 138, 158, 168]. Some studies have also explored the relationships within various input data sources [43, 71, 85].

Failure Category Classification. This task addresses the challenge of determining the type of anomaly the system is experiencing. This task, often formulated as a multi-class classification problem, categorizes anomalies into predefined types, such as CPU resource shortages, memory constraints, or software configuration errors. Traditional failure category classification typically requires predefined failure categories and trains multi-class models strictly according to these

categories. As a result, the effectiveness of these models is limited by the predefined categories. However, the advent of large language models has expanded the scope of this task. Many studies now attempt to directly construct appropriate prompts and incorporate external knowledge, enabling LLMs to autonomously identify failure categories [16, 106, 117, 151, 155, 159, 168, 177, 180]. Other works have pre-trained foundation models for time-series or log data, allowing the addition of new categories with minimal fine-tuning on the existing model [34, 38, 84, 123, 126, 174].

Root Cause Report Generation. This task emerges with the enhanced natural language generation and reasoning capabilities of large language models. This development goes beyond isolated tasks such as failure localization and failure category classification. Instead, there is a growing focus on creating comprehensive and more easily understandable root cause reports. These reports not only include information on failure location and category classification but also provide detailed reasoning about the underlying cause of the failure [2, 29, 43, 109, 109, 133, 145, 170]. This comprehensive approach significantly aids system maintenance personnel in analyzing and resolving issues more efficiently.

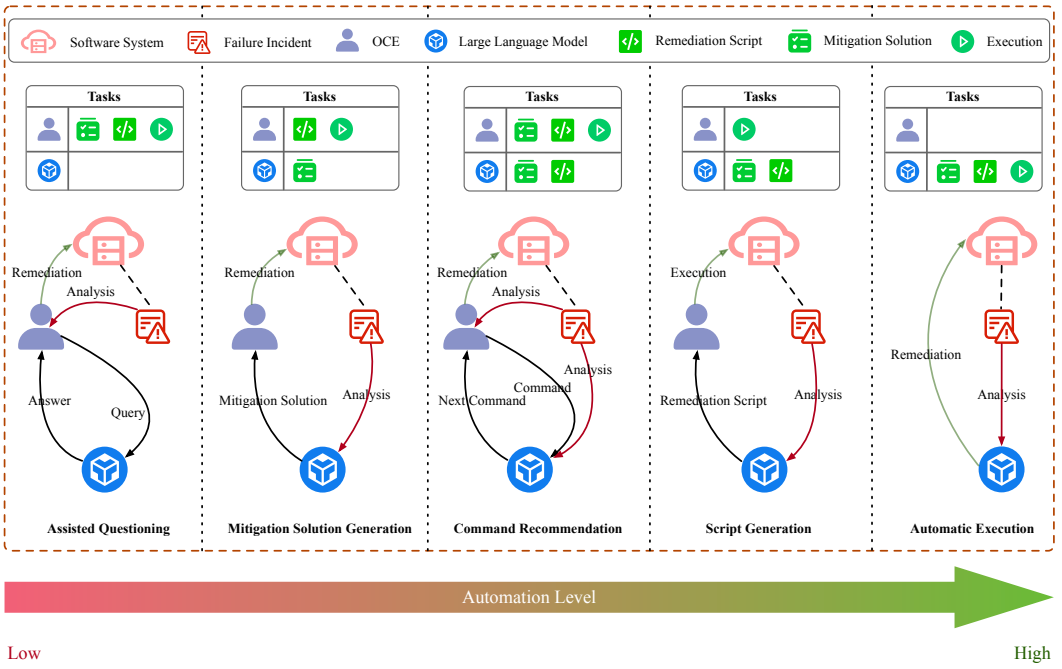


Fig. 9. Various Types of Auto Remediation Approaches

4.3 Assisted Remediation

After identifying the type and location of the software system anomaly, the next step is to automatically mitigate and resolve the issue based on this information [17]. Before the advent of large language models, the automation level of these remediation tasks was relatively low. However, with the introduction of LLMs, the degree of automation has significantly improved. As illustrated in Figure 9, this survey classifies assisted remediation approaches by automation level into five categories, arranged by increasing levels of automation: Assisted Questioning, Mitigation Solution Generation, Command Recommendation, Script Generation, and Automatic Execution. Among

these, only Mitigation Solution Generation existed in traditional assisted remediation approaches, while the other tasks have emerged as new possibilities enabled by LLMs. These automation levels were determined through a combination of literature review and expert interviews. First, we summarized the entities involved in AIOps based on relevant papers published by the Microsoft research team [14, 16, 50, 117, 126, 145, 163, 164, 170]. Using these entities, we identify and categorize the relevant literature for our review. Subsequently, we conduct interviews with OCEs from partner companies to evaluate and refine our proposed assisted remediation framework. Unlike earlier AIOps tasks, this stage does not rely on a single primary data source. Instead, it integrates all data modalities discussed in RQ1. For example, Assisted Questioning primarily relies on historical QA data; Mitigation Solution Generation may leverage incident reports, traces, metrics, and logs—often enriched with software information used as a knowledge base; Command Recommendation and Script Generation, on the other hand, typically depend on source code and configuration data.

Assisted Questioning. This task involves using LLMs to assist operations personnel by answering system-related questions. By allowing operations staff to directly query the LLM software, detailed responses can be obtained quickly, speeding up the resolution of software system failures [32, 45, 79, 102, 166]. This type of task emerged with the rise of LLMs, particularly GPT-3.5.

Mitigation Solution Generation. In this task, LLMs generate potential mitigation solutions for the detected anomalies. These solutions are often derived from large datasets of historical incident reports and resolutions, providing actionable suggestions to operations personnel for addressing issues [2, 29, 36, 78, 91, 117, 128, 129]. Before LLMs, this subtask primarily involved incident triage [148], but these methods are not intelligent. Mitigation solution generation appears more advanced and tailored.

Command Recommendation. This task leverages LLMs to recommend pre-existing scripts that can be used to remediate the detected issue. The system can suggest the next command to enter when operations staff input commands (e.g., shell commands), thus shortening the repair time [119, 179]. This method gains popularity with the advent of LLMs.

Script Generation. This task skips the command recommendation step, directly using LLMs to generate custom scripts tailored to resolve specific detected anomalies. This involves creating new scripts based on the details of the issue and the context provided by system logs and metrics, allowing for more precise and effective remediation actions [3, 50, 87, 95, 103, 113, 132].

Automatic Execution. The highest level of automation involves LLMs not only generating the necessary remediation scripts but also executing them automatically. This end-to-end process ensures that detected anomalies are addressed without manual intervention, significantly speeding up resolution time and reducing the workload on operations personnel. Although this method is highly attractive, related work is limited, and its practical effectiveness remains to be verified [9, 59, 64, 98, 114].

5 RQ3: LLM-BASED METHODS FOR AIOPS

In this section, we introduce various LLM-based methods and explore the specific domains in which they are frequently applied to different AIOps tasks. Additionally, we analyze the respective advantages and limitations of applying these approaches.

Many surveys have proposed various LLM-based approaches [12, 31, 70, 90, 111], many of which have been applied to address AIOps tasks. As shown in Figure 10, in this survey, we categorize these approaches into five groups: foundation model, fine-tuning approach, embedding-based approach, prompt-based approach, and knowledge-based approach. The foundation model refers to pre-trained language models that serve as the base for AIOps tasks without further modifications. The fine-tuning approach involves adapting these pre-trained models to specific tasks through additional training on task-specific data. The embedding-based approach uses the representations

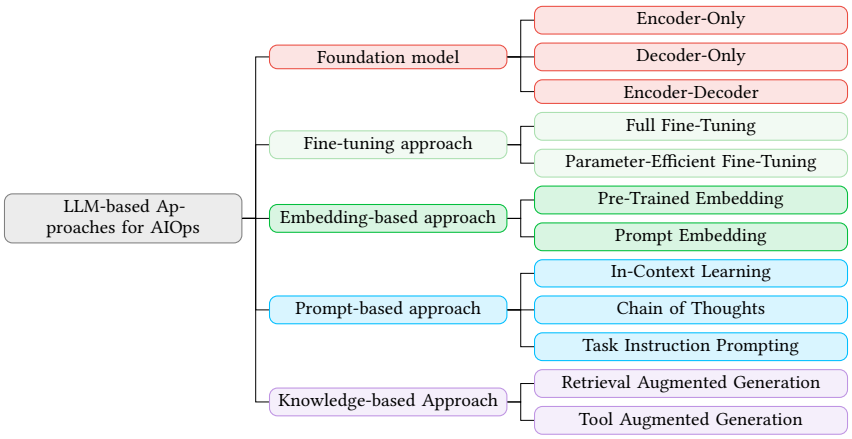


Fig. 10. LLM-based Approaches for AIOps

generated by pre-trained models to capture semantic information and improve task performance. The prompt-based approach leverages natural language prompts to guide the model's responses, enabling it to perform specific tasks based on the given instructions. The knowledge-based approach integrates external knowledge sources to enhance the model's capabilities by providing relevant context or executing specialized tools for more accurate and informed AIOps task completion.

5.1 Foundation Model

As mentioned earlier, in the era of large language models, an increasing number of efforts focus on enhancing model generalization, with one of the most straightforward approaches being the development of foundation models. Currently, almost all large language models are based on the Transformer architecture, with differences arising from the utilization of encoder and decoder blocks [144]. Specifically, the encoder maps the input sequence into latent representations, while the decoder processes these representations to generate the output sequence. Consequently, these works can be categorized into three architectural types: **encoder-only**, **decoder-only**, and **encoder-decoder**. In the context of AIOps tasks, such models are predominantly applied in anomaly detection and failure category classification, commonly referred to as metric and log foundation models.

Encoder-Only Models. Encoder-only architectures focus on processing the entire input sequence simultaneously, extracting contextual information for predictions. These models generally have smaller parameter sizes and were more prominent in the earlier stages of large language model development. Despite their reduced popularity in foundation model pretraining, they remain effective for specific applications requiring fine-grained input analysis.

LoFI [44] employs an encoder-only design with a prompt-based tuning method to extract detailed insights from log data, particularly for failure diagnosis tasks. Similarly, ART [126] uses an encoder-only framework to pretrain a system that integrates anomaly detection, failure triage, and root cause localization, focusing on metrics data. These examples demonstrate the continued relevance of encoder-only models in domains where efficiency and detailed input analysis are prioritized.

Decoder-Only Models. Decoder-only architectures are characterized by autoregressive token generation, where each token depends on previous ones. Popularized by models like GPT-3, these architectures typically involve larger parameter sizes and are well-suited for generative tasks.

Several studies have adapted decoder-only architectures for AIOps. Lag-Llama [107] pretrains a foundation model for univariate probabilistic time series forecasting using lags as covariates

within a decoder-only transformer framework. TimesFM [19] employs a patched-decoder-style attention model to robustly address varying forecasting history lengths, prediction horizons, and temporal granularities. For domain-specific tasks, ShellGPT [119] adapts a GPT architecture to align shell scripting with natural language, embedding domain knowledge for automation tasks. Timer [81] builds a GPT-style model tailored for long-term sequence modeling (LTSMs), combining forecasting, imputation, and anomaly detection in a unified framework. These works highlight the generative power and flexibility of decoder-only models for complex, sequential tasks.

Encoder-Decoder Models. Encoder-decoder architectures integrate both encoder and decoder blocks, allowing distinct phases for input processing and output generation. This design is particularly well-suited for tasks requiring intricate input-output mappings. Foundational examples such as T5 and BART epitomize this architecture, balancing parameter size for flexibility and efficiency.

In AIOps, only a few studies leverage this architecture. TimeGPT [72] employs a deep encoder-decoder structure to pretrain a foundation model for time series data, focusing on extracting complex patterns from over 100 billion data points [144]. Similarly, SimMTM [22] reconstructs masked time points in metrics data by leveraging the encoder-decoder structure, which aggregates information from neighboring points. This method excels at recovering temporal information outside the manifold. For log analytics, PreLog [67] uses this architecture with entry-level and sequence-level objectives to train a model effective in log parsing and log-based anomaly detection. Moreover, KAD-Disformer [146] adapts an encoder-decoder transformer for disentangling temporal and spatial dynamics in metrics data, enhancing anomaly detection performance. These works highlight the versatility of encoder-decoder models in capturing diverse patterns across time-series and log-based data.

5.2 Fine-tuning Approach

Directly applying general foundation models to AIOps tasks often does not yield optimal results. Therefore, many AIOps approaches fine-tune these models using domain-specific datasets. This fine-tuning can be categorized into two types: **full fine-tuning** and **parameter-efficient fine-tuning**.Most of these methods primarily rely on metrics and log data. However, a smaller subset of approaches also incorporates QA data and source code during the fine-tuning process.

Full Fine-Tuning. Full fine-tuning updates all parameters of a foundation model with domain-specific data, enabling the model to adapt comprehensively to new tasks. However, due to the computational complexity and large parameter sizes of models like GPT-3.5 and GPT-4, most current efforts focus on relatively smaller models with fewer than 100 billion parameters, such as Codex and LLaMA.

For instance, PromptCast [141] fine-tunes text-visible models such as T5 and BART with carefully designed prompts to enable forecasting-based anomaly detection [144]. Similarly, UniTime [77] adopts a traditional fine-tuning approach on GPT-2 to develop a unified model for cross-domain time series forecasting. These works demonstrate the efficacy of full fine-tuning in adapting general-purpose models to time-series-specific tasks.

Other works explore innovative fine-tuning methodologies. AnomalyLLM [76] leverages LLaMA2-7B and introduces a dynamic-aware contrastive fine-tuning technique to address graph-based anomaly detection. Meanwhile, RAG4ITOps [166] uses a Qwen-14B-base model and implements Continue Retrieval Augmented Fine-Tuning to build an assisted questioning system tailored to IT operations.

Parameter-Efficient Fine-Tuning. Parameter-efficient fine-tuning offers a more computationally feasible alternative by updating only a subset of model parameters. This approach includes techniques such as layer-freezing, adapter tuning, and task-conditional fine-tuning, which are particularly effective in resource-constrained environments.

Several studies demonstrate the utility of parameter-efficient methods for AIOps. Komal et al. [113] investigate the performance of in-context tuning and few-shot tuning on LLaMA2-70B for auto-remediation tasks, highlighting the adaptability of large language models with minimal updates. Similarly, Toufique et al. [2] conduct a large-scale empirical study on models like GPT-Neo and GPT-2, evaluating their effectiveness in assisting engineers with root cause analysis and production incident mitigation through few-shot tuning.

A common strategy in this domain involves using adapters to improve fine-tuning efficiency. OWL [32] applies a mixture-of-adapters strategy to LLaMA2-13B, achieving significant improvements across tasks like assisted questioning and log-based anomaly detection. Similarly, LLM4TS [11] focuses on freezing the majority of GPT-2 parameters and fine-tuning only a small subset for general forecasting-based anomaly detection.

Instruction tuning has also emerged as a promising parameter-efficient approach. LogLM [78] fine-tunes LLaMA2-7B using instruction-based tasks, enabling it to handle a wide range of log analysis tasks such as log parsing and anomaly detection. Additionally, models like CrashEventLLM [93], Fatemeh et al. [35], and Hongwei et al. [52] explore supervised fine-tuning on models like GPT-3 and LLaMA2-7B to optimize log-based anomaly detection.

Few-shot tuning has been particularly effective for log parsing and analysis. For example, LLM-Parser [86] and Maryam et al. [88] evaluate few-shot tuning on models such as Flan-T5-small, Flan-T5-base, LLaMA-7B, ChatGLM-6B, and Mistral-7B-Instruct, demonstrating the ability of light-weight fine-tuning to adapt general models for specialized tasks.

5.3 Embedding-based Approach

Many data sources in AIOps, such as logs and documentation, contain rich semantic information, making embeddings a crucial component in capturing and representing this information in a structured format. Embedding-based approaches for AIOps can be broadly categorized into two types: **pre-trained embedding** and **prompt embedding**. Pre-trained embeddings leverage pre-existing large language models (LLMs) to extract semantic representations directly, while prompt embeddings involve designing task-specific prompts to activate the LLMs' capabilities for specialized data processing such as metric data.

Pre-Trained Embedding. Pre-trained embeddings rely on leveraging representations generated by models like GPT or T5, which have been trained on extensive corpora and can encode rich semantic information. These embeddings are versatile and can be applied to various AIOps tasks without further fine-tuning, making them particularly popular for tasks involving log analysis.

For instance, Harold et al. [99] utilized embeddings from GPT-2, chaining these embeddings through time using a recurrent neural network (BiLSTM) to model and learn normal system behaviors. Similarly, Egil et al. [56] employed GPT-2-based embeddings within an autoencoding framework, compressing the representations and applying self-supervised learning for log-based anomaly detection. Setu et al. [115] focused on GPT-3, utilizing its word embeddings and tokenizer to transform log data into a format suitable for identifying unusual patterns and anomalies effectively.

Prompt Embedding. Prompt embedding works focus on designing specific embedding methods tailored for LLMs to effectively capture semantic information from various data sources in AIOps. By providing natural language prompts or instructions, these methods activate the LLM's processing capability to generate task-specific embeddings. This approach enables flexible and task-specific embeddings suitable for different AIOps applications. Currently, many works based on metrics utilize this approach to transform metrics data into a format more suitable for LLM understanding.

Time-LLM [53] proposes Prompt-as-Prefix (PaP), which enriches the input context and directs the transformation of reprogrammed input patches. The transformed time series patches from the LLM are then projected to obtain the forecasts. TEST [123] builds an encoder based on metrics data

to embed them by instance-wise, feature-wise, and text-prototype-aligned contrast, and then creates prompts to make the LLM more receptive to embeddings, achieving excellent results in failure category classification. Nate et al. [30] used prompt embedding techniques with GPT-3, LLaMA-2, and GPT-4 to accomplish anomaly detection and metrics imputation tasks. TEMPO [10] decouples complex time series into trend, seasonal, and residual components, mapping them to corresponding latent spaces to create inputs recognizable by GPT. To achieve this, TEMPO constructs a prompt pool and assigns different prompts to different decoupled components, enabling the model to adapt to changes in the time series distribution using historical information.

5.4 Prompt-based Approach

In addition to leveraging the embedding capabilities of large language models, many AIOps approaches directly utilize prompt-based interaction with LLMs to perform various tasks. These methods can be broadly categorized into In-Context Learning (ICL), Chain-of-Thought (CoT) reasoning, and Task Instruction Prompting. In practice, these techniques are often used in combination to complement one another. For simplicity, we classify such hybrid methods based on the primary technique emphasized in each study. These approaches commonly utilize diverse data sources, including log data, incident reports, and metrics data.

In-Context Learning (ICL). In-Context Learning involves providing LLMs with examples or context within the prompt to guide the model in task execution. This approach allows the model to infer patterns from the examples and produce outputs aligned with desired results, making it particularly useful for tasks requiring format adherence or specific outputs.

Many studies utilize ICL in log parsing tasks. Works such as LILAC [51], LLMParser [86], Lemur [33], DivLog [140], LogParser-LLM [172], ECLIPSE [167], Free [136], LogBatcher [137], and LogPPT [66] effectively pair ICL with cache mechanisms to improve parsing accuracy and efficiency. For log-based anomaly detection, LogPrompt [80], LogEval [18], and Semirald [125] adopt ICL to guide the model in identifying anomalous patterns within structured or semi-structured data.

In the context of incident reports, ICL has been widely used to address tasks like root cause analysis. For instance, RCACopilot [16] and Xpert [50] leverage ICL to structure diagnostic processes, ensuring accurate identification of causative factors. For metrics-based methods, LasRCA [38] combines one-shot learning with smaller models as classifiers to enhance metrics-based anomaly detection. Despite its focus on metrics, the method integrates incident reports for failure identification, showcasing the adaptability of ICL in leveraging both structured and unstructured data across various AIOps scenarios. Similarly, AnomalyLLM [76] employs a previously fine-tuned LLaMA2-7B model and applies ICL to effectively detect metrics-based anomalies, further demonstrating the versatility of this approach.

Chain of Thoughts (CoT). Chain of Thought prompting enhances LLMs' capabilities in multi-step reasoning tasks by structuring prompts to guide the model through intermediate steps before arriving at the final output. This method is particularly advantageous for tasks requiring logical decomposition or multi-step processing.

For time-series forecasting, LSTPrompt [75] decomposes predictions into short-term and long-term sub-tasks, utilizing CoT techniques to generate tailored prompts for each step. Similarly, LogGPT [80] leverages CoT prompting to refine anomaly detection in logs, enabling the model to better capture nuanced semantic features.

In causal discovery and root cause analysis, RealTCD [71] applies CoT prompting to identify causative relationships between events. LM-PACE [151] uses CoT techniques to enhance GPT-4's diagnostic capabilities in analyzing incident reports, while [36] extends this approach to generating mitigation solutions from incident reports. These examples highlight the effectiveness of CoT prompting in addressing complex analytical tasks within AIOps.

Task Instruction Prompting. Task Instruction Prompting involves providing explicit, detailed instructions to LLMs to guide task execution. This method is particularly effective for zero-shot or simple tasks where direct instructions suffice to produce satisfactory results.

Early works in AIOps often relied on this approach for anomaly detection tasks. For instance, LogGPT [105] and Priyanka et al. [106] conduct studies on zero-shot log-based anomaly detection using ChatGPT, revealing its limited performance without further fine-tuning or specialized techniques. Metrics-based anomaly detection has also been tackled through instruction-based approaches. Sigllm[98] and TabLLM [2] integrate time-series-to-text conversion modules and explicitly prompt LLMs to identify anomalies within the data.

In root cause analysis and assisted remediation tasks, Achraf et al. [98] demonstrate the use of zero-shot prompt-based script generation to support solution development, while Toufique et al. [2] compared zero-shot prompting with parameter-efficient tuning for root cause analysis and solution recommendation based on incident reports. Andres et al. [106] employe GPT-3.5 in a zero-shot setup for log-based root cause analysis, underscoring the flexibility and adaptability of task instruction prompting in diverse applications.

5.5 Knowledge-based Approach

In addition to directly utilizing prompts for AIOps tasks, knowledge-based approaches enhance the performance of LLMs by integrating historical data or external knowledge sources. These approaches can be broadly categorized into two main types: Retrieval-Augmented Generation (RAG) and Tool-Augmented Generation (TAG). Methods based on historical data typically leverage log data, metrics data, and incident reports. In contrast, methods that rely on external knowledge sources often incorporate trace data, system information, and source code to enrich the model's reasoning capabilities.

Retrieval Augmented Generation (RAG). Retrieval Augmented Generation enhances the model's outputs by retrieving relevant information from external knowledge bases or databases. This approach enables LLMs to incorporate up-to-date or domain-specific knowledge that is not part of their pre-training. In AIOps, RAG is widely applied to access historical logs, incident reports, or technical documentation, helping models provide contextually informed and reliable solutions.

Some works employ RAG specifically to retrieve historical data, using it as a reference for current AIOps tasks. These methods can be considered an improvement on the aforementioned ICL-based approaches, leveraging specialized similarity retrieval techniques to select the most relevant "shots" for the large language model. Xuchao et al. [170] developed an incident report summarization system that converts report content into a 768-dimensional dense vector space. In production environments, similar vectors are retrieved based on proximity to the current incident and passed to GPT-4 for root cause analysis. LM-PACE [151] uses similarity retrieval to identify relevant historical incidents, jointly retrieving corresponding recommended root causes to assist in confidence calibration. Xpert [50] retrieves similar incident reports to recommend repair commands. Oasis [54] generates summaries of current incidents by retrieving historical patterns between incidents. Methods such as RAGLog [100] and LogRAG [169] use historical log retrieval to enhance log-based anomaly detection. MonitorAssistant [145] retrieves similar historical monitor metrics and anomaly labels to support anomaly detection.

Other works utilize RAG to retrieve external knowledge, providing richer data and domain expertise to strengthen AIOps solutions. Drishti et al. [29] retrieve information across different stages of the software development lifecycle (SDLC) to enhance the LLM's root cause analysis capabilities. LLMDB [180] and DB-GPT [179] integrate database-specific knowledge (e.g., developer documentation and user manuals) to optimize anomaly detection and diagnosis in database systems. LogExpert [128] retrieves domain features similar to anomalous logs to aid in generating mitigation

solutions. RAG4ITOps [166] retrieves domain-specific knowledge bases from cloud computing to support IT operations knowledge Q&A.

Tool Augmented Generation (TAG). Tool Augmented Generation equips large language models with the ability to interact with external tools, APIs, or software systems to assist in completing tasks. In AIOps, TAG-based methods enable LLMs to query monitoring systems, execute diagnostic scripts, or use debugging tools, generating more accurate and actionable results. This approach is particularly effective for automating complex workflows that demand precise execution and real-time data interaction.

In AIOps, some TAG-based methods use tools to aid in failure identification and classification. RCAgent [133] utilizes information-gathering and analytical tools to perform root cause analysis for Apache Flink. RCACopilot [16] focuses exclusively on information-gathering tools to assist in incident report-based root cause analysis.

Other works employ tools to execute specific operations, which are primarily applied in assisted remediation tasks. Charles et al. [9] introduce a GPT-based AI agent capable of competently using 150 distinct tools across nine categories, from file manipulation to programming compilation, to manage Linux servers. NetLM [129] leverages various traffic control tools to automate network management. Pitikorn et al. [59] employ code execution tools to automatically execute Python scripts, enabling autonomous cloud system repairs. ChatOps4Msa [132], Pesl et al. [103], and Marco et al. [3] utilize DevOps tools to explore automated service composition.

A subset of works integrates both analytical and execution tools, blending their capabilities. Honghao et al. [118] propose a method that combines analytical and execution tools to perform early warning, troubleshooting, and repair tasks for AI clusters. LLexus [64] integrates script tools and semantic tools based on troubleshooting guides (TSGs) to mitigate failures in cloud systems. AIOpsLab [117] leverages traces, metrics, and log data in combination with various information-gathering and automated execution tools to localize and resolve failures in microservice clusters.

6 RQ4: EVALUATING LLM-BASED AIOPS

In this section, we discuss the emergence of new evaluation methodologies in the era of large language models for AIOps. These advancements are reflected in two key areas: the development of new evaluation metrics and the creation of novel datasets.

6.1 Emerging Evaluation Metrics

With the evolution of AIOps tasks in the LLM era, the range of evaluation metrics has expanded significantly. As shown in Figure 11, current evaluation metrics can be categorized into four types: classification task metrics, generation task metrics, execution task metrics, and manual evaluation. Notably, the latter three have emerged as new additions in the LLM era.

6.1.1 Classification Task Metrics. This category of metrics originates from traditional AIOps tasks and continues to be utilized in some works in the era of large language models (LLMs) [10, 11, 19, 29, 30, 33, 51, 53, 66, 72, 75, 100, 107, 123, 140]. These metrics can be broadly classified into two types: **Performance Metrics** and **Error Metrics**.

Performance Metrics. These metrics evaluate the accuracy of model classifications, including precision, recall, F1-score, accuracy, and AUC-ROC. They are frequently used for tasks such as log-based anomaly detection and failure category classification. In failure localization tasks, metrics like Acc@ (accuracy of the top-N predictions) are also employed. For log parsing, additional metrics such as grouping accuracy (GA) are utilized to measure performance.

Error Metrics. These metrics are primarily used for regression tasks, quantifying the difference between predicted and actual values. Examples include MAE (Mean Absolute Error), MSE (Mean

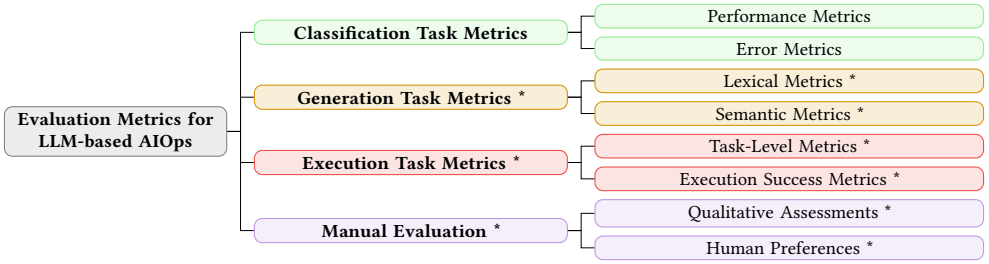


Fig. 11. Evaluation Metrics for LLM-based AIOps (Metrics marked with * are new metrics that have emerged in the era of LLMs)

Squared Error), RMSE (Root Mean Squared Error), and MAPE (Mean Absolute Percentage Error). Such metrics are commonly applied in evaluating large time-series models for tasks like failure prediction and anomaly detection.

6.1.2 Generation Task Metrics. This category of metrics was not frequently used in traditional AIOps. However, with the integration of large language models, a variety of generative tasks have emerged, such as root cause report generation and mitigation solution generation. As a result, generation-specific metrics are now essential for evaluating these methods [2, 50, 78, 89, 93, 95, 109, 119, 128, 133, 170]. These metrics can be classified into two groups: **Lexical Metrics** and **Semantic Metrics**.

Lexical Metrics. These metrics focus on surface-level textual similarity, comparing the generated output with reference texts based on word overlap. Examples include BLEU (Bilingual Evaluation Understudy), ROUGE (Recall-Oriented Understudy for Gisting Evaluation), and METEOR (Metric for Evaluation of Translation with Explicit ORdering). These metrics are particularly useful for tasks where adherence to specific language or terminology is critical, such as generating standardized incident reports or summaries.

Semantic Metrics. These metrics assess the meaning and contextual relevance of the generated text rather than its exact lexical similarity. Examples include BERTScore, BLEURT (Bilingual Evaluation Understudy with Representations from Transformers) and NUBIA (NeUral Based Interchangeability Assessor). These metrics are crucial for evaluating tasks where the primary concern is the meaningfulness or contextual appropriateness of the output, such as mitigation suggestions or explanations for system failures.

6.1.3 Execution Task Metrics. This category of metrics has emerged primarily with the rise of large language models, as they are designed to evaluate assisted remediation tasks, particularly the effectiveness of script generation and automatic execution [9, 91, 112–114]. These metrics can be broadly divided into two groups: Task-Level Metrics and Execution Success Metrics.

Task-Level Metrics. These metrics assess the quality and correctness of individual tasks or blocks within a generated script. They are particularly important for evaluating script generation tasks, where the output consists of multiple functional components that need to be validated independently. Examples include:

- **Functional Correctness (FC)**: Measures whether each generated task or script successfully fulfills its intended functionality. This often involves running the tasks and verifying the results against ground truth or predefined benchmarks.
- **Average Correctness (AC)**: Evaluates the proportion of successfully executed tasks within a generated script. For example, in systems like Ansible, tasks represent units of work such

as installing a package, configuring services, or copying files. The AC metric calculates the average success rate of all tasks, reflecting the overall correctness of the script.

Execution Success Metrics. These metrics evaluate the success of executing the generated script or code in its entirety, ensuring the intended outcome is achieved. They focus on operational effectiveness, particularly for automation tasks. Examples include:

- **Execution Success Rate**: Measures the percentage of generated scripts or workflows that execute without errors and achieve the desired outcome.
- **Correct Refactorings**: Assesses whether modifications or refinements to an existing script or workflow (e.g., bug fixes or performance optimizations) lead to successful and improved executions.

6.1.4 Manual Evaluation. While the previously discussed metrics are mathematically rigorous and provide objective assessments, they are often insufficient to fully evaluate LLM-based AIOps methods. The powerful generative capabilities of large language models frequently require human involvement to assess their outputs [64, 94, 98, 109, 116, 117, 145, 151]. This is especially true for tasks such as root cause report generation and mitigation solution generation, where the outputs often lack fixed labels or have inherently flexible ground truth. Manual evaluation methods can be broadly categorized into two groups: **Qualitative Assessments** and **Human Preferences**.

Qualitative Assessments. .These assessments involve human experts objectively evaluating the results against predefined qualitative grading criteria. The goal is to identify specific shortcomings and categorize the outputs based on quality or error types. For exmaple, a root cause report can be evaluted by the following levels [109]:

- Precise: Results are accurate, relevant, and fully aligned with the task requirements.
- Imprecise: Results are partially correct but contain minor inaccuracies or missing details.
- Hallucination: Results include fabricated or irrelevant information not grounded in the input data.
- Reasoning Error: Logical errors or invalid inferences made in the generated output.
- Retrieval Error: Incorrect or incomplete use of retrieved information during generation.

Human Preferences. This type of manual evaluation focuses on subjective assessments of the utility, relevance, and overall usefulness of the generated results. Unlike strict pre-defined criteria, evaluators provide judgments based on personal perspectives or case-specific requirements. A classic example is case studies, where human evaluators assess outputs in real-world scenarios to determine whether the generated solutions are practically useful and actionable.

6.2 Development of Novel Datasets

In the era of large language models, several novel datasets have been specifically introduced for AIOps. It is important to note that this section focuses exclusively on works that explicitly design datasets to cater to large language models. In other words, we exclude datasets proposed in recent years that, although utilized in LLM-based works, were not originally developed with a specific focus on LLMs. Notably, most of these datasets are concentrated on assisted remediation tasks due to the abundance of new challenges in this area.

Only a few works have introduced datasets targeting failure prediction and root cause analysis tasks. For example, LogEval [18] was proposed to evaluate LLM capabilities across various log analysis tasks, including log parsing, log anomaly detection, log fault diagnosis, and log summarization. It assesses these tasks using 4,000 publicly available log data entries and employs 15 different prompts for each task.

In the assisted question task, OpsEval [79] is introduced as a comprehensive task-oriented benchmark designed specifically for LLMs. This dataset includes 7,184 multiple-choice questions and 1,736 question-answering (QA) formats in both English and Chinese. It represents the first effort to evaluate LLMs' proficiency across various critical scenarios and skill levels. Additionally, OpsEval provides an online leaderboard and a continuously updated dataset, along with the leaderboard itself [2]. Similarly, OWL-bench [32] has established a dataset for the operation and maintenance (O&M) domain. This dataset spans nine O&M-related subdomains, showcasing the diversity and hierarchical structure of LLM capabilities within the O&M field.

In the script generation task, KubePlaybook [95] introduced a curated dataset comprising 130 natural language prompts for generating automation-focused remediation code scripts.

Additionally, some works have proposed benchmark frameworks encompassing the entire AIOps lifecycle. For instance, AIOpsLab [14], developed by Microsoft, provides a prototype implementation leveraging an agent-cloud interface. This framework orchestrates applications, injects real-time failures using chaos engineering, and interfaces with agents to localize and resolve failures.

7 CHALLENGES AND FUTURE DIRECTIONS

While large language models have significantly enhanced AIOps, numerous challenges still remain to be addressed. Many of these challenges stem from inherent limitations of LLMs themselves, such as hallucinations, inconsistent outputs, context limitations, and others. These general issues have been widely studied in the field of natural language processing (NLP) and fall outside the scope of this paper.

However, beyond these general challenges, LLM-based AIOps presents a set of unique and domain-specific issues that require further investigation. These include the time-efficiency and cost-effectiveness of LLM-based AIOps solutions, the in-depth utilization of diverse failure data sources, the generalizability and adaptability of models during software evolution, and the integration with existing AIOps toolchains. These domain-specific challenges will be discussed in detail in the following sections.

7.1 Time-Efficiency and Cost-effectiveness of LLM-based AIOps Solutions

Large language models require substantial computational resources and energy, leading to high operational costs. The training and deployment of LLMs involve extensive use of GPUs or TPUs, making them less accessible for many organizations. Additionally, the inference costs associated with LLMs can be prohibitive, especially when real-time or near-real-time responses are required. This poses significant hurdles, particularly for small to medium-sized enterprises or in scenarios where computational resources are constrained.

Among the tasks of AIOps, the computational overhead of LLMs is most critical for failure perception tasks. These steps theoretically need to run continuously during the operation of a software system and require high real-time performance. For example, if the anomaly detection time window is set to 10 seconds, meaning an anomaly detection process is triggered every 10 seconds, the anomaly detection model must infer results within 1 second to promptly notify OCEs in case of anomalies. Some non-LLM-based approaches have focused on addressing this issue [49, 74], but currently, no LLM-based work has adequately tackled this problem. However, this issue is becoming increasingly significant in the LLM era.

On the other hand, while the higher computational efficiency and cost associated with LLMs may be acceptable for root cause analysis and auto remediation (since these steps do not need to run continuously within the software system), this issue still requires careful consideration.

[2]https://opseval.cstcloud.cn/content/leaderboard

Excessive computational costs could potentially make LLM-based approaches less effective than using small-scale models in combination with OCEs. For example, a well-designed smaller model augmented by human expertise might achieve comparable results at a fraction of the cost and resource consumption, making it a more practical and scalable solution in certain contexts.

In summary, balancing computational cost and performance is crucial for the application of LLMs in AIOps. If the cost of running LLMs outweighs their benefits, organizations may find it more advantageous to leverage traditional models or hybrid approaches that combine the strengths of smaller-scale models and human oversight. Therefore, future research should focus on optimizing the computational efficiency of LLMs for AIOps, possibly by exploring methods that organically combine small-scale models, large language models, and OCEs. This integrated approach could provide a time-efficient and cost-effective solution while maximizing the strengths of each component.

7.2 In-depth Usage of More Diverse Failure Data Sources

Several important data sources have yet to be utilized in LLM-based approaches for AIOps. As illustrated in Section 3, there are three crucial types of system-generated data in software system runtime: metrics, logs, and traces. While there has been significant progress in utilizing metrics and logs, no current work has effectively incorporated traces data.

Traces data is particularly valuable as it provides insights into the calls between nodes in a software system cluster and the interactions within internal components. The lack of LLM-based approaches using traces data can be attributed to its high complexity and volume, making it challenging to integrate trace data with LLMs. Traces often involve detailed and nested sequences of events that need to be represented in a manner that LLMs can effectively understand and utilize. However, traces data remains a crucial resource for comprehensive system monitoring and AIOps. Future research should focus on developing methods to effectively integrate traces data into LLM-based AIOps methods.

On the other hand, while many failure perception works have been based on logs, a significant portion of these works utilize pre-trained models like T5 and GPT-2, which are not particularly large in scale [44, 67, 126, 141]. These approach result in relatively high training costs. Only a small fraction of works have utilized larger models such as GPT-3.5, but these studies are primarily based on simple datasets [24, 100, 105]. For these datasets, failure perception can often be achieved with traditional machine learning models [63, 157], which diminishes the perceived superiority of LLM-based methods. Therefore, future research should explore more effective ways to apply logs to LLM-based approaches. One feasible solution could be prompt embedding, which has been widely used in metrics-based work.

Lastly, in LLM-based root cause analysis, many works have used incident reports as the primary data source and achieved excellent results [2, 29, 36, 50, 54, 59, 62, 109, 151, 170]. However, this approach disrupts the automation flow of AIOps, as root cause analysis should be triggered by failure perception. Therefore, future work should focus on using system-generated data to trigger failure perception, generate corresponding incident reports, and then apply root cause analysis methods to analyze these generated incident reports.

7.3 Generalizability and Model Adaptability in Software Evolution

A key challenge in traditional ML- and DL-based approaches to AIOps lies in significant performance degradation when models are applied to different software systems or encounter modifications to the original system. Techniques such as meta-learning and online learning have sought to mitigate this by enabling models to retrain efficiently with minimal new data, thus restoring their effectiveness in evolving environments.

In contrast, LLM-based approaches, leveraging the inferential power of large language models, are expected to exhibit high generalizability and adaptability due to extensive pre-training on diverse textual data. However, empirical validation of this expectation remains limited. While some foundation models demonstrate cross-platform effectiveness, many works, particularly those using prompt-based methods, lack systematic evaluation of generalizability and adaptability in real-world software evolution scenarios.

To address this gap, future research must emphasize rigorous testing of LLMs' ability to generalize across diverse systems and maintain or recover performance amid software changes. This involves designing systematic experiments to simulate varying software environments and assessing model resilience through incremental fine-tuning or continuous learning paradigms. Furthermore, exploring the intersection of advanced LLM architectures, transfer learning techniques, and adaptive learning algorithms will be pivotal in ensuring robust, scalable, and adaptable solutions for AIOps.

7.4 Integration with Existing AIOps Toolchains

Current LLM-based AIOps solutions predominantly focus on constructing entirely new methods, often disregarding the value of existing smaller model-based approaches. However, considering the diverse tasks within AIOps, such as failure prediction, root cause analysis, and assisted remediation, a more effective approach would involve integrating LLMs with existing toolchains to enhance overall performance.

In failure prediction, smaller models or rule-based systems could quickly process high-frequency, structured data like logs or metrics, providing initial anomaly detection. LLMs, with their contextual reasoning and flexibility, could then interpret and correlate these anomalies with broader system behaviors, offering deeper insights. Similarly, in root cause analysis, traditional log parsers or anomaly detectors might identify potential fault patterns, which LLMs could augment by generating detailed reports or explanations that draw on diverse data sources and historical knowledge.

In assisted remediation, existing automation frameworks (e.g., Ansible or Kubernetes operators) could handle script execution and simple task orchestration, while LLMs contribute by generating, validating, or refining automation scripts. This division of labor ensures efficiency, as routine or repetitive tasks are offloaded to smaller models or tools, while LLMs focus on tasks requiring advanced reasoning or generation.

Future research should focus on developing interoperable frameworks that seamlessly integrate LLMs with smaller models, legacy systems, and specialized tools. Such modular integration not only leverages the strengths of traditional methods but also reduces the computational burden and operational risks of relying solely on large models—ensuring scalable, cost-efficient, and easily adoptable solutions across diverse AIOps scenarios.

8 CONCLUSION

LLMs are bringing significant changes to the field of AIOps. Their powerful generative capabilities fundamentally reshape many AIOps tasks and methodologies. In this survey, we analyzed the emerging utilization of LLMs for AIOps, focusing on research related to LLMs (models with more than 1 billion parameters). We examined the diverse failure data sources applied in this field, including advanced LLM-based processing techniques for legacy data and the utilization of new data sources enabled by LLMs (RQ1). We also investigated the evolving tasks in AIOps with LLMs, emphasizing the emergence of new tasks and analyzing the distribution of publications across different tasks (RQ2). Following this, we explored the various LLM-based methods applied to these tasks (RQ3). Lastly, we reviewed the evaluation methodologies in AIOps adapted to the integration of LLMs (RQ4).

Despite the progress made, several areas require further exploration. These include the time-efficiency and cost-effectiveness of LLM-based AIOps solutions, deeper utilization of diverse data sources, generalizability and model adaptability during software evolution, integration with existing AIOps toolchains.

In conclusion, the realm of AIOps in the era of LLMs is a dynamically advancing field teeming with both promise and challenges. Its progression is vital to underpinning the stability and reliability of software systems. By addressing the challenges outlined in this survey, LLM-powered methodologies for AIOps are poised for broader, more impactful real-world implementations. This advancement will not only enhance the resilience and efficiency of modern software ecosystems but also pave the way for intelligent, adaptive, and future-ready solutions to manage software failures effectively.

ACKNOWLEDGMENTS

This work was supported by the National Key R&D Research Fund of China (2021YFF0704202).

REFERENCES

[1] Arman Ahmed, K Sadanandan Sajan, Anurag Srivastava, and Yinghui Wu. 2021. Anomaly detection, localization and classification using drifting synchrophasor data streams. *IEEE Transactions on Smart Grid* 12, 4 (2021), 3570–3580.

[2] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending root-cause and mitigation steps for cloud incidents using large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1737–1749.

[3] Marco Aiello and Ilche Georgievski. 2023. Service composition in the ChatGPT era. *Service Oriented Computing and Applications* 17, 4 (2023), 233–238.

[4] Khalid Ayedh Alharthi, Arshad Jhumka, Sheng Di, and Franck Cappello. 2022. Clairvoyant: a log-based transformer-decoder for failure prediction in large-scale systems. In *Proceedings of the 36th ACM International Conference on Supercomputing*. 1–14.

[5] Khalid Ayed Alharthi, Arshad Jhumka, Sheng Di, Lin Gui, Franck Cappello, and Simon McIntosh-Smith. 2023. Time machine: generative real-time model for failure (and lead time) prediction in hpc systems. In *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 508–521.

[6] Sarah Alnegheimish, Linh Nguyen, Laure Berti-Equille, and Kalyan Veeramachaneni. 2024. Large language models can be zero-shot anomaly detectors for time series? *arXiv preprint arXiv:2405.14755* (2024).

[7] Dharun Anandayuvaraj, Matthew Campbell, Arav Tewari, and James C Davis. 2024. FAIL: Analyzing Software Failures from the News Using LLMs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 506–518.

[8] Bhavya Bhavya, Paulina Toro Isaza, Yu Deng, Michael Nidd, Amar Prakash Azad, Larisa Shwartz, and ChengXiang Zhai. 2023. Exploring Large Language Models for Low-Resource IT Information Extraction. In *2023 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 1203–1212.

[9] Charles Cao, Feiyi Wang, Lisa Lindley, and Zejiang Wang. 2024. Managing Linux servers with LLM-based AI agents: An empirical evaluation with GPT4. *Machine Learning with Applications* 17 (2024), 100570.

[10] Defu Cao, Furong Jia, Sercan O Arik, Tomas Pfister, Yixiang Zheng, Wen Ye, and Yan Liu. 2024. TEMPO: Prompt-based Generative Pre-trained Transformer for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations*.

[11] Ching Chang, Wen-Chih Peng, and Tien-Fu Chen. 2023. Llm4ts: Two-stage fine-tuning for time-series forecasting with pre-trained llms. *arXiv preprint arXiv:2308.08469* (2023).

[12] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.

[13] Georgios Chatzigeorgakidis, Konstantinos Lentzos, and Dimitrios Skoutas. 2024. MultiCast: Zero-Shot Multivariate Time Series Forecasting Using LLMs. In *2024 IEEE 40th International Conference on Data Engineering Workshops (ICDEW)*. IEEE, 119–127.

[14] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. [n.d.]. AIOPSLAB: AHOLISTIC FRAMEWORK TO EVALUATE AI AGENTS FOR ENABLING AUTONOMOUS CLOUDS. ([n. d.]).

[15] Yakun Chen, Xianzhi Wang, and Guandong Xu. 2023. Gatgpt: A pre-trained large language model with graph attention network for spatiotemporal imputation. *arXiv preprint arXiv:2311.14332* (2023).

[16] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. 2024. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 674–688.

[17] Qian Cheng, Doyen Sahoo, Amrita Saha, Wenzhuo Yang, Chenghao Liu, Gerald Woo, Manpreet Singh, Silvio Saverese, and Steven CH Hoi. 2023. Ai for it operations (aiops) on cloud platforms: Reviews, opportunities and challenges. *arXiv preprint arXiv:2304.04661* (2023).

[18] Tianyu Cui, Shiyu Ma, Ziang Chen, Tong Xiao, Shimin Tao, Yilun Liu, Shenglin Zhang, Duoming Lin, Changchang Liu, Yuzhe Cai, et al. 2024. Logeval: A comprehensive benchmark suite for large language models in log analysis. *arXiv preprint arXiv:2407.01896* (2024).

[19] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A decoder-only foundation model for time-series forecasting. In *Forty-first International Conference on Machine Learning*.

[20] Josu Diaz-de Arcaya, Juan López-de Armentia, Gorka Zárate, and Ana I Torre-Bastida. 2024. Towards the self-healing of Infrastructure as Code projects using constrained LLM technologies. In *Proceedings of the 5th ACM/IEEE International Workshop on Automated Program Repair*. 22–25.

[21] Josu Diaz-De-Arcaya, Ana I Torre-Bastida, Gorka Zarate, Raul Minon, and Aitor Almeida. 2023. A joint study of the challenges, opportunities, and roadmap of mlops and aiops: A systematic survey. *Comput. Surveys* 56, 4 (2023), 1–30.

[22] Jiaxiang Dong, Haixu Wu, Haoran Zhang, Li Zhang, Jianmin Wang, and Mingsheng Long. 2024. Simmtm: A simple pre-training framework for masked time-series modeling. *Advances in Neural Information Processing Systems* 36 (2024).

[23] Chiming Duan, Tong Jia, Yong Yang, Guiyang Liu, Jinbu Liu, Huxing Zhang, Qi Zhou, Ying Li, and Gang Huang. 2025. EagerLog: Active Learning Enhanced Retrieval Augmented Generation for Log-based Anomaly Detection. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.

[24] Chris Egersdoerfer, Di Zhang, and Dong Dai. 2023. Early exploration of using chatgpt for log-based anomaly detection on parallel file systems logs. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*. 315–316.

[25] Vijay Ekambaram, Arindam Jati, Nam H Nguyen, Pankaj Dayama, Chandra Reddy, Wesley M Gifford, and Jayant Kalagnanam. 2024. TTMs: Fast Multi-level Tiny Time Mixers for Improved Zero-shot and Few-shot Forecasting of Multivariate Time Series. *arXiv preprint arXiv:2401.03955* (2024).

[26] Stephen Elliot. 2014. DevOps and the cost of downtime: Fortune 1000 best practice metrics quantified. *International Data Corporation (IDC)* (2014).

[27] Asma Fariha, Vida Gharavian, Masoud Makrehchi, Shahryar Rahnamayan, Sanaa Alwidian, and Akramul Azim. 2024. Log Anomaly Detection by Leveraging LLM-Based Parsing and Embedding with Attention Mechanism. In *2024 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*. IEEE, 859–863.

[28] Azul Garza and Max Mergenthaler-Canseco. 2023. TimeGPT-1. *arXiv preprint arXiv:2310.03589* (2023).

[29] Drishti Goel, Fiza Husain, Aditya Singh, Supriyo Ghosh, Anjaly Parayil, Chetan Bansal, Xuchao Zhang, and Saravan Rajmohan. 2024. X-lifecycle learning for cloud incident management using llms. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 417–428.

[30] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew G Wilson. 2024. Large language models are zero-shot time series forecasters. *Advances in Neural Information Processing Systems* 36 (2024).

[31] Jindong Gu, Zhen Han, Shuo Chen, Ahmad Beirami, Bailan He, Gengyuan Zhang, Ruotong Liao, Yao Qin, Volker Tresp, and Philip Torr. 2023. A systematic survey of prompt engineering on vision-language foundation models. *arXiv preprint arXiv:2307.12980* (2023).

[32] Hongcheng Guo, Jian Yang, Jiaheng Liu, Liqun Yang, Linzheng Chai, Jiaqi Bai, Junran Peng, Xiaorong Hu, Chao Chen, Dongfeng Zhang, et al. 2024. OWL: A Large Language Model for IT Operations. In *The Twelfth International Conference on Learning Representations*.

[33] Hongcheng Guo, Wei Zhang, Anjie Le, Jian Yang, Jiaheng Liu, Zhoujun Li, Tieqiao Zheng, Shi Xu, Runqiang Zang, Liangfan Zheng, et al. 2024. Lemur: Log Parsing with Entropy Sampling and Chain-of-Thought Merging. *arXiv preprint arXiv:2402.18205* (2024).

[34] Pranjal Gupta, Harshit Kumar, Debanjana Kar, Karan Bhukar, Pooja Aggarwal, and Prateeti Mohapatra. 2023. Learning Representations on Logs for AIOps. In *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*. IEEE, 155–166.

[35] Fatemeh Hadadi, Qinghua Xu, Domenico Bianculli, and Lionel Briand. 2024. Anomaly Detection on Unstable Logs with GPT Models. *arXiv preprint arXiv:2406.07467* (2024).

[36] Pouya Hamadanian, Behnaz Arzani, Sadjad Fouladi, Siva Kesava Reddy Kakarla, Rodrigo Fonseca, Denizcan Billor, Ahmad Cheema, Edet Nkposong, and Ranveer Chandra. 2023. A Holistic View of AI-driven Network Incident Management. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*. 180–188.

[37] Shangbin Han, Qianhong Wu, Han Zhang, Bo Qin, Jiankun Hu, Xingang Shi, Linfeng Liu, and Xia Yin. 2021. Log-based anomaly detection with robust feature extraction and online learning. *IEEE Transactions on Information Forensics and Security* 16 (2021), 2300–2311.

[38] Yongqi Han, Qingfeng Du, Ying Huang, Jiaqi Wu, Fulong Tian, and Cheng He. 2024. The Potential of One-Shot Failure Root Cause Analysis: Collaboration of the Large Language Model and Small Classifier. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 931–943.

[39] Ahatsham Hayat and Mohammad R Hasan. 2025. A Context-Aware Approach for Enhancing Data Imputation with Pre-trained Language Models. In *Proceedings of the 31st International Conference on Computational Linguistics*. 5668–5685.

[40] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. 2024. LLMeLog: An Approach for Anomaly Detection based on LLM-enriched Log Events. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 132–143.

[41] Shilin He, Jieming Zhu, Pinjia He, and Michael R Lyu. 2023. Loghub: A large collection of system log datasets towards automated log analytics. (2023).

[42] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. Tabllm: Few-shot classification of tabular data with large language models. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 5549–5581.

[43] Adha Hrusto, Per Runeson, and Magnus C Ohlsson. 2024. Autonomous monitors for detecting failures early and reporting interpretable alerts in cloud operations. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. 47–57.

[44] Junjie Huang, Zhihan Jiang, Jinyang Liu, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Cong Feng, Hui Dong, Zengyin Yang, and Michael R Lyu. 2024. Demystifying and Extracting Fault-indicating Information from Logs for Failure Diagnosis. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 511–522.

[45] Shaohan Huang, Yi Liu, Jiaxing Qi, Jing Shang, Zhiwen Xiao, Carol Fung, Zhihui Wu, Hailong Yang, Zhongzhi Luan, and Depei Qian. 2024. Gloss: Guiding Large Language Models to Answer Questions from System Logs. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 91–101.

[46] Azam Ikram, Sarthak Chakraborty, Subrata Mitra, Shiv Saini, Saurabh Bagchi, and Murat Kocaoglu. 2022. Root cause analysis of failures in microservices through causal discovery. *Advances in Neural Information Processing Systems* 35 (2022), 31158–31170.

[47] Michel Jacobsen and Marina Tropmann-Frick. 2024. Imputation Strategies in Time Series Based on Language Models. *Datenbank-Spektrum* 24, 3 (2024), 197–207.

[48] Yuhe JI, Jing HAN, Yongxin ZHAO, Shenglin ZHANG, and Zican GONG. 2023. Log Anomaly Detection Through GPT-2 for Large Scale Systems. *ZTE Communications* 21, 3 (2023), 70.

[49] Tong Jia, Yifan Wu, Chuanjia Hou, and Ying Li. 2021. Logflash: Real-time streaming anomaly detection and diagnosis from system logs for large-scale software systems. In *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 80–90.

[50] Yuxuan Jiang, Chaoyun Zhang, Shilin He, Zhihao Yang, Minghua Ma, Si Qin, Yu Kang, Yingnong Dang, Saravan Rajmohan, Qingwei Lin, et al. 2024. Xpert: Empowering incident management with query recommendations via large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[51] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R Lyu. 2024. Lilac: Log parsing using llms with adaptive parsing cache. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 137–160.

[52] Hongwei Jin, George Papadimitriou, Krishnan Raghavan, Pawel Zuk, Prasanna Balaprakash, Cong Wang, Anirban Mandal, and Ewa Deelman. 2024. Large Language Models for Anomaly Detection in Computational Workflows: From Supervised Fine-Tuning to In-Context Learning. In *2024 SC24: International Conference for High Performance Computing, Networking, Storage and Analysis SC*. IEEE Computer Society, 1466–1482.

[53] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. 2023. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728* (2023).

[54] Pengxiang Jin, Shenglin Zhang, Minghua Ma, Haozhe Li, Yu Kang, Liqun Li, Yudong Liu, Bo Qiao, Chaoyun Zhang, Pu Zhao, et al. 2023. Assess and summarize: Improve outage understanding with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1657–1668.

[55] Yuyuan Kang, Xiangdong Huang, Shaoxu Song, Lingzhe Zhang, Jialin Qiao, Chen Wang, Jianmin Wang, and Julian Feinauer. 2022. Separation or not: On handing out-of-order time-series data in leveled lsm-tree. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 3340–3352.

[56] Egil Karlsen, Xiao Luo, Nur Zincir-Heywood, and Malcolm Heywood. 2024. Large language models and unsupervised feature learning: implications for log analysis. *Annals of Telecommunications* (2024), 1–19.

[57] Khalid S Khan, Regina Kunz, Jos Kleijnen, and Gerd Antes. 2003. Five steps to conducting a systematic review. *Journal of the royal society of medicine* 96, 3 (2003), 118–121.

[58] Subina Khanal, Seshu Tirupathi, Giulio Zizzo, Ambrish Rawat, and Torben Bach Pedersen. 2024. Domain Adaptation for Time series Transformers using One-step fine-tuning. *arXiv preprint arXiv:2401.06524* (2024).

[59] Pitikorn Khlaisamniang, Prachaya Khomduean, Kriangkrai Saetan, and Supasin Wonglapsuwan. 2023. Generative AI for self-healing systems. In *2023 18th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*. IEEE, 1–6.

[60] Donghyun Kim, Sriram Ravula, Taemin Ha, Alexandros G Dimakis, Daehyeok Kim, and Aditya Akella. 2024. Large Language Models as Realistic Microservice Trace Generators. *arXiv preprint arXiv:2502.17439* (2024).

[61] Barbara Kitchenham, O Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic literature reviews in software engineering–a systematic literature review. *Information and software technology* 51, 1 (2009), 7–15.

[62] Jinxi Kuang, Jinyang Liu, Junjie Huang, Renyi Zhong, Jiazhen Gu, Lan Yu, Rui Tan, Zengyin Yang, and Michael R Lyu. 2024. Knowledge-aware Alert Aggregation in Large-scale Cloud Systems: a Hybrid Approach. *arXiv preprint arXiv:2403.06485* (2024).

[63] Max Landauer, Florian Skopik, and Markus Wurzenberger. 2023. A Critical Review of Common Log Data Sets Used for Evaluation of Sequence-based Anomaly Detection Techniques. *arXiv preprint arXiv:2309.02854* (2023).

[64] Pedro Las-Casas, Alok Gautum Kumbhare, Rodrigo Fonseca, and Sharad Agarwal. 2024. LLexus: an AI agent system for incident management. *ACM SIGOPS Operating Systems Review* 58, 1 (2024), 23–36.

[65] Van-Hoang Le and Hongyu Zhang. 2021. Log-based anomaly detection without log parsing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 492–504.

[66] Van-Hoang Le and Hongyu Zhang. 2023. Log Parsing: How Far Can ChatGPT Go?. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1699–1704.

[67] Van-Hoang Le and Hongyu Zhang. 2024. PreLog: A Pre-trained Model for Log Analytics. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.

[68] Guoliang Li, Xuanhe Zhou, Ji Sun, Xiang Yu, Yue Han, Lianyuan Jin, Wenbo Li, Tianqing Wang, and Shifu Li. 2021. opengauss: An autonomous database system. *Proceedings of the VLDB Endowment* 14, 12 (2021), 3028–3042.

[69] Hongbo Li, Wenli Zheng, Feilong Tang, Yanmin Zhu, and Jielong Huang. 2023. Few-shot time-series anomaly detection with unsupervised domain adaptation. *Information Sciences* 649 (2023), 119610.

[70] Junyi Li, Tianyi Tang, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2024. Pre-trained language models for text generation: A survey. *Comput. Surveys* 56, 9 (2024), 1–39.

[71] Peiwen Li, Xin Wang, Zeyang Zhang, Yuan Meng, Fang Shen, Yue Li, Jialong Wang, Yang Li, and Wenweu Zhu. 2024. LLM-Enhanced Causal Discovery in Temporal Domain from Interventional Data. *arXiv preprint arXiv:2404.14786* (2024).

[72] Wenlong Liao, Fernando Porte-Agel, Jiannong Fang, Christian Rehtanz, Shouxiang Wang, Dechang Yang, and Zhe Yang. 2024. TimeGPT in Load Forecasting: A Large Time Series Model Perspective. *arXiv preprint arXiv:2404.04885* (2024).

[73] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuewei Chen. 2016. Log clustering based problem identification for online service systems. In *Proceedings of the 38th International Conference on Software Engineering Companion*. 102–111.

[74] Yifei Lin, Hanqiu Deng, and Xingyu Li. 2024. FastLogAD: Log Anomaly Detection with Mask-Guided Pseudo Anomaly Generation and Discrimination. *arXiv preprint arXiv:2404.08750* (2024).

[75] Haoxin Liu, Zhiyuan Zhao, Jindong Wang, Harshavardhan Kamarthi, and B Aditya Prakash. 2024. LSTPrompt: Large Language Models as Zero-Shot Time Series Forecasters by Long-Short-Term Prompting. *arXiv preprint arXiv:2402.16132* (2024).

[76] Shuo Liu, Di Yao, Lanting Fang, Zhetao Li, Wenbin Li, Kaiyu Feng, XiaoWen Ji, and Jingping Bi. 2024. AnomalyLLM: Few-shot Anomaly Edge Detection for Dynamic Graphs using Large Language Models. *arXiv preprint arXiv:2405.07626* (2024).

[77] Xu Liu, Junfeng Hu, Yuan Li, Shizhe Diao, Yuxuan Liang, Bryan Hooi, and Roger Zimmermann. 2024. Unitime: A language-empowered unified model for cross-domain time series forecasting. In *Proceedings of the ACM on Web Conference 2024*. 4095–4106.

[78] Yilun Liu, Yuhe Ji, Shimin Tao, Minggui He, Weibin Meng, Shenglin Zhang, Yongqian Sun, Yuming Xie, Boxing Chen, and Hao Yang. 2024. LogLM: From Task-based to Instruction-based Automated Log Analysis. *arXiv preprint arXiv:2410.09352* (2024).

[79] Yuhe Liu, Changhua Pei, Longlong Xu, Bohan Chen, Mingze Sun, Zhirui Zhang, Yongqian Sun, Shenglin Zhang, Kun Wang, Haiming Zhang, et al. 2023. OpsEval: A Comprehensive Task-Oriented AIOps Benchmark for Large Language Models. *arXiv preprint arXiv:2310.07637* (2023).

[80] Yilun Liu, Shimin Tao, Weibin Meng, Jingyu Wang, Wenbing Ma, Yuhang Chen, Yanqing Zhao, Hao Yang, and Yanfei Jiang. 2024. Interpretable online log analysis using large language models with prompt strategies. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 35–46.

[81] Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. 2024. Timer: Generative Pre-trained Transformers Are Large Time Series Models. In *Forty-first International Conference on Machine Learning*.

[82] Yingzhe Lyu, Heng Li, Zhen Ming, Ahmed E Hassan, et al. 2023. Assessing the maturity of model maintenance techniques for AIOps solutions. *arXiv preprint arXiv:2311.03213* (2023).

[83] A Hamou-Lhadj M Mehrabi and H Moosavi. 2024. The Effectiveness of Compact Fine-Tuned LLMs in Log Parsing. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 97–109.

[84] Lipeng Ma, Weidong Yang, Sihang Jiang, Ben Fei, Mingjie Zhou, Shuhao Li, Bo Xu, and Yanghua Xiao. 2024. LUK: Empowering Log Understanding with Expert Knowledge from Large Language Models. *arXiv preprint arXiv:2409.01909* (2024).

[85] Lipeng Ma, Weidong Yang, Bo Xu, Sihang Jiang, Ben Fei, Jiaqing Liang, Mingjie Zhou, and Yanghua Xiao. 2024. Knowlog: Knowledge enhanced pre-trained language model for log understanding. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[86] Zeyang Ma, An Ran Chen, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. 2024. LLMParser: An Exploratory Study on Using Large Language Models for Log Parsing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[87] Ehud Malul, Yair Meidan, Dudu Mimran, Yuval Elovici, and Asaf Shabtai. 2024. GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation. *arXiv preprint arXiv:2405.19954* (2024).

[88] Maryam Mehrabi, Abdelwahab Hamou-Lhadj, and Hossein Moosavi. 2024. The Effectiveness of Compact Fine-Tuned LLMs in Log Parsing. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 1–12.

[89] Weibin Meng, Federico Zaiter, Yuzhe Zhang, Ying Liu, Shenglin Zhang, Shimin Tao, Yichen Zhu, Tao Han, Yongpeng Zhao, En Wang, et al. 2023. Logsummary: Unstructured log summarization for software systems. *IEEE Transactions on Network and Service Management* 20, 3 (2023), 3803–3815.

[90] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large language models: A survey. *arXiv preprint arXiv:2402.06196* (2024).

[91] Francesco Minna, Fabio Massacci, and Katja Tuma. 2024. Analyzing and Mitigating (with LLMs) the Security Misconfigurations of Helm Charts from Artifact Hub. *arXiv preprint arXiv:2403.09537* (2024).

[92] Panagiotis Misiakos, Chris Wendler, and Markus Püschel. 2024. Learning DAGs from data with few root causes. *Advances in Neural Information Processing Systems* 36 (2024).

[93] Priyanka Mudgal, Bijan Arbab, and Swaathi Sampath Kumar. 2024. CrashEventLLM: Predicting System Crashes with Large Language Models. In *2024 International Conference on Information Technology and Computing (ICITCOM)*. IEEE, 72–76.

[94] Priyanka Mudgal and Rita Wouhaybi. 2023. An Assessment of ChatGPT on Log Data. In *International Conference on AI-generated Content*. Springer, 148–169.

[95] Zakeya Namrud, Komal Sarda, Marin Litoiu, Larisa Shwartz, and Ian Watts. 2024. KubePlaybook: A Repository of Ansible Playbooks for Kubernetes Auto-Remediation with LLMs. In *Companion of the 15th ACM/SPEC International Conference on Performance Engineering*. 57–61.

[96] Giang Nguyen, Stefan Dlugolinsky, Viet Tran, and Álvaro López García. 2024. Network security AIOps for online stream data monitoring. *Neural Computing and Applications* 36, 24 (2024), 14925–14949.

[97] Paolo Notaro, Jorge Cardoso, and Michael Gerndt. 2021. A survey of aiops methods for failure management. *ACM Transactions on Intelligent Systems and Technology (TIST)* 12, 6 (2021), 1–45.

[98] Achraf Othman, Amira Dhouib, and Aljazi Nasser Al Jabor. 2023. Fostering websites accessibility: A case study on the use of the Large Language Models ChatGPT for automatic remediation. In *Proceedings of the 16th International Conference on PErvasive Technologies Related to Assistive Environments*. 707–713.

[99] Harold Ott, Jasmin Bogatinovski, Alexander Acker, Sasho Nedelkoski, and Odej Kao. 2021. Robust and transferable anomaly detection in log data using pre-trained language models. In *2021 IEEE/ACM international workshop on cloud intelligence (CloudIntelligence)*. IEEE, 19–24.

[100] Jonathan Pan, Wong Swee Liang, and Yuan Yidi. 2024. Raglog: Log anomaly detection using retrieval augmented generation. In *2024 IEEE World Forum on Public Safety Technology (WFPST)*. IEEE, 169–174.

[101] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton Van Den Hengel. 2021. Deep learning for anomaly detection: A review. *ACM computing surveys (CSUR)* 54, 2 (2021), 1–38.

[102] Gijun Park and Dohoon Kim. 2023. Formulating an Korean LLM-Based Interactive Assistant for Enhanced IT Collaboration in Microservice Environments. In *2023 IEEE 8th International Conference on Smart Cloud (SmartCloud)*. IEEE, 176–181.

[103] Robin D Pesl, Miles Stötzner, Ilche Georgievski, and Marco Aiello. 2023. Uncovering LLMs for Service-Composition: Challenges and Opportunities. In *International Conference on Service-Oriented Computing*. Springer, 39–48.

[104] Pankaj Prasad and Charley Rich. 2018. Market guide for aiops platforms. *Retrieved March* 12, 2020 (2018), 2–9.

[105] Jiaxing Qi, Shaohan Huang, Zhongzhi Luan, Shu Yang, Carol Fung, Hailong Yang, Depei Qian, Jing Shang, Zhiwen Xiao, and Zhihui Wu. 2023. Loggpt: Exploring chatgpt for log-based anomaly detection. In *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. IEEE, 273–280.

[106] Andres Quan, Leah Howell, and Hugh Greenberg. 2023. Heterogeneous Syslog Analysis: There Is Hope. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. 581–587.

[107] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Arian Khorasani, George Adamopoulos, Rishika Bhagwatkar, Marin Biloš, Hena Ghonia, Nadhir Hassen, Anderson Schneider, et al. 2023. Lag-llama: Towards foundation models for time series forecasting. In *R0-FoMo: Robustness of Few-shot and Zero-shot Learning in Large Foundation Models*.

[108] Youcef Remil, Anes Bendimerad, Romain Mathonat, and Mehdi Kaytoue. 2024. Aiops solutions for incident management: Technical guidelines and a comprehensive literature review. *arXiv preprint arXiv:2404.01363* (2024).

[109] Devjeet Roy, Xuchao Zhang, Rashi Bhave, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring LLM-based Agents for Root Cause Analysis. *arXiv preprint arXiv:2403.04123* (2024).

[110] Alicia Russell-Gilbert, Alexander Sommers, Andrew Thompson, Logan Cummins, Sudip Mittal, Shahram Rahimi, Maria Seale, Joseph Jaboure, Thomas Arnold, and Joshua Church. 2024. AAD-LLM: Adaptive Anomaly Detection Using Large Language Models. *arXiv preprint arXiv:2411.00914* (2024).

[111] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. *arXiv preprint arXiv:2402.07927* (2024).

[112] Komal Sarda. 2023. Leveraging Large Language Models for Auto-remediation in Microservices Architecture. In *2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*. IEEE, 16–18.

[113] Komal Sarda, Zakeya Namrud, Marin Litoiu, Larisa Shwartz, and Ian Watts. 2024. Leveraging Large Language Models for the Auto-remediation of Microservice Applications: An Experimental Study. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 358–369.

[114] Komal Sarda, Zakeya Namrud, Raphael Rouf, Harit Ahuja, Mohammadreza Rasolroveicy, Marin Litoiu, Larisa Shwartz, and Ian Watts. 2023. Adarma auto-detection and auto-remediation of microservice anomalies by leveraging large language models. In *Proceedings of the 33rd Annual International Conference on Computer Science and Software Engineering*. 200–205.

[115] Jahanggir Hossain Setu, Md Shazzad Hossain, Nabarun Halder, Ashraful Islam, and M Ashraful Amin. 2024. Optimizing Software Release Management with GPT-Enabled Log Anomaly Detection. In *International Conference on Pattern Recognition*. Springer, 351–365.

[116] Shiwen Shan, Yintong Huo, Yuxin Su, Yichen Li, Dan Li, and Zibin Zheng. 2024. Face It Yourselves: An LLM-Based Two-Stage Strategy to Localize Configuration Errors via Logs. *arXiv preprint arXiv:2404.00640* (2024).

[117] Manish Shetty, Yinfang Chen, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Xuchao Zhang, Jonathan Mace, Dax Vandevoorde, Pedro Las-Casas, Shachee Mishra Gupta, et al. 2024. Building AI Agents for Autonomous Clouds: Challenges and Design Principles. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*. 99–110.

[118] Honghao Shi, Longkai Cheng, Wenli Wu, Yuhang Wang, Xuan Liu, Shaokai Nie, Weixv Wang, Xuebin Min, Chunlei Men, and Yonghua Lin. 2024. Enhancing Cluster Resilience: LLM-agent Based Autonomous Intelligent Cluster Diagnosis System and Evaluation Framework. *arXiv preprint arXiv:2411.05349* (2024).

[119] Jie Shi, Sihang Jiang, Bo Xu, Jiaqing Liang, Yanghua Xiao, and Wei Wang. 2023. ShellGPT: Generative Pre-trained Transformer Model for Shell Language Understanding. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 671–682.

[120] Jacopo Soldani and Antonio Brogi. 2022. Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey. *ACM Computing Surveys (CSUR)* 55, 3 (2022), 1–39.

[121] Jing Su, Chufeng Jiang, Xin Jin, Yuxin Qiao, Tingsong Xiao, Hongda Ma, Rong Wei, Zhi Jing, Jiajun Xu, and Junhong Lin. 2024. Large Language Models for Forecasting and Anomaly Detection: A Systematic Literature Review. *arXiv preprint arXiv:2402.10350* (2024).

[122] Yicheng Sui, Yuzhe Zhang, Jianjun Sun, Ting Xu, Shenglin Zhang, Zhengdan Li, Yongqian Sun, Fangrui Guo, Junyu Shen, Yuzhi Zhang, et al. 2023. LogKG: Log Failure Diagnosis through Knowledge Graph. *IEEE Transactions on*

Services Computing (2023).

[123] Chenxi Sun, Hongyan Li, Yaliang Li, and Shenda Hong. 2024. TEST: Text Prototype Aligned Embedding to Activate LLM's Ability for Time Series. In *The Twelfth International Conference on Learning Representations*.

[124] Yuchen Sun, Yanpiao Chen, Haotian Zhao, and Shan Peng. 2023. Design and Development of a Log Management System Based on Cloud Native Architecture. In *2023 9th International Conference on Systems and Informatics (ICSAI)*. IEEE, 1–6.

[125] Yicheng SUN, Jacky Keung, Zhen Yang, Shuo Liu, and Hi Kuen Yu. 2024. Semirald: A Semi-Supervised Hybrid Language Model for Robust Anomalous Log Detection. *Available at SSRN 4927951* (2024).

[126] Yongqian Sun, Binpeng Shi, Mingyu Mao, Minghua Ma, Sibo Xia, Shenglin Zhang, and Dan Pei. 2024. ART: A Unified Unsupervised Framework for Incident Management in Microservice Systems. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1183–1194.

[127] Łukasz Tulczyjew, Kinan Jarrah, Charles Abondo, Dina Bennett, and Nathanael Weill. 2024. LLMcap: Large Language Model for Unsupervised PCAP Failure Detection. In *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 1559–1565.

[128] Jiabo Wang, Guojun Chu, Jingyu Wang, Haifeng Sun, Qi Qi, Yuanyi Wang, Ji Qi, and Jianxin Liao. 2024. LogExpert: Log-based Recommended Resolutions Generation using Large Language Model. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 42–46.

[129] Jingyu Wang, Lei Zhang, Yiran Yang, Zirui Zhuang, Qi Qi, Haifeng Sun, Lu Lu, Junlan Feng, and Jianxin Liao. 2023. Network Meets ChatGPT: Intent Autonomous Management, Control and Operation. *Journal of Communications and Information Networks* 8, 3 (2023), 239–255.

[130] Lu Wang, Chaoyun Zhang, Ruomeng Ding, Yong Xu, Qihang Chen, Wentao Zou, Qingjun Chen, Meng Zhang, Xuedong Gao, Hao Fan, et al. 2023. Root cause analysis for microservice systems via hierarchical reinforcement learning from human feedback. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5116–5125.

[131] Qing Wang, Wubai Zhou, Chunqiu Zeng, Tao Li, Larisa Shwartz, and Genady Ya Grabarnik. 2017. Constructing the knowledge base for cognitive it service management. In *2017 IEEE International Conference on Services Computing (SCC)*. IEEE, 410–417.

[132] Sheng-Kai Wang, Shang-Pin Ma, Chen-Hao Chao, and Guan-Hong Lai. 2023. Low-code ChatOps for Microservices Systems Using Service Composition. In *2023 IEEE International Conference on e-Business Engineering (ICEBE)*. IEEE, 55–62.

[133] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. Rcagent: Cloud root cause analysis by autonomous agents with tool-augmented large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4966–4974.

[134] Xinjie Wei, Jie Wang, Chang-ai Sun, Dave Towey, Shoufeng Zhang, Wanqing Zuo, Yiming Yu, Ruoyi Ruan, and Guyang Song. 2024. Log-based anomaly detection for distributed systems: State of the art, industry experience, and open issues. *Journal of Software: Evolution and Process* (2024).

[135] Jinfeng Wen, Zhenpeng Chen, Federica Sarro, Zixi Zhu, Yi Liu, Haodi Ping, and Shangguang Wang. 2024. LLM-Based Misconfiguration Detection for AWS Serverless Computing. *arXiv preprint arXiv:2411.00642* (2024).

[136] Yi Xiao, Van-Hoang Le, and Hongyu Zhang. 2024. Free: Towards More Practical Log Parsing with Large Language Models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 153–165.

[137] Yi Xiao, Van-Hoang Le, and Hongyu Zhang. 2024. Stronger, Faster, and Cheaper Log Parsing with LLMs. *arXiv preprint arXiv:2406.06156* (2024).

[138] Zhiqiang Xie, Yujia Zheng, Lizi Ottens, Kun Zhang, Christos Kozyrakis, and Jonathan Mace. 2024. Cloud Atlas: Efficient Fault Localization for Cloud Systems using Language Models and Causal Insight. *arXiv preprint arXiv:2407.08694* (2024).

[139] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2023. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms. *arXiv preprint arXiv:2306.13063* (2023).

[140] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. 2024. DivLog: Log Parsing with Prompt Enhanced In-Context Learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.

[141] Hao Xue and Flora D Salim. 2023. Promptcast: A new prompt-based learning paradigm for time series forecasting. *IEEE Transactions on Knowledge and Data Engineering* (2023).

[142] Senming Yan, Lei Shi, Jing Ren, Wei Wang, Yaxin Liu, Limin Sun, Xiong Wang, and Wei Zhang. 2024. Log-Based Anomaly Detection with Transformers Pre-Trained on Large-Scale Unlabeled Data. In *ICC 2024-IEEE International Conference on Communications*. IEEE, 1–6.

[143] Fangkai Yang, Wenjie Yin, Lu Wang, Tianci Li, Pu Zhao, Bo Liu, Paul Wang, Bo Qiao, Yudong Liu, Mårten Björkman, et al. 2023. Diffusion-Based Time Series Data Imputation for Cloud Failure Prediction at Microsoft 365. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2050–2055.

[144] Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. 2024. A Survey of Time Series Foundation Models: Generalizing Time Series Representation with Large Language Model. *ArXiv* abs/2405.02358 (2024). https://api.semanticscholar.org/CorpusID:269605992

[145] Zhaoyang Yu, Minghua Ma, Chaoyun Zhang, Si Qin, Yu Kang, Chetan Bansal, Saravan Rajmohan, Yingnong Dang, Changhua Pei, Dan Pei, et al. 2024. Monitorassistant: Simplifying cloud service monitoring via large language models. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 38–49.

[146] Zhaoyang Yu, Changhua Pei, Xin Wang, Minghua Ma, Chetan Bansal, Saravan Rajmohan, Qingwei Lin, Dongmei Zhang, Xidao Wen, Jianhui Li, et al. 2024. Pre-trained kpi anomaly detection model through disentangled transformer. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6190–6201.

[147] Yue Yuan, Wenchang Shi, Bin Liang, and Bo Qin. 2019. An approach to cloud execution failure diagnosis based on exception logs in openstack. In *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 124–131.

[148] Chunqiu Zeng, Wubai Zhou, Tao Li, Larisa Shwartz, and Genady Ya Grabarnik. 2017. Knowledge guided hierarchical multi-label classification over ticket data. *IEEE Transactions on Network and Service Management* 14, 2 (2017), 246–260.

[149] Zhengran Zeng, Yuqun Zhang, Yong Xu, Minghua Ma, Bo Qiao, Wentao Zou, Qingjun Chen, Meng Zhang, Xu Zhang, Hongyu Zhang, et al. 2023. Traceark: Towards actionable performance anomaly alerting for online service systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 258–269.

[150] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning. In *Proceedings of the 44th International Conference on Software Engineering*. 623–634.

[151] Dylan Zhang, Xuchao Zhang, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. LM-PACE: Confidence estimation by large language models for effective root causing of cloud incidents. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 388–398.

[152] Ke Zhang, Chenxi Zhang, Xin Peng, and Chaofeng Sha. 2022. Putracead: Trace anomaly detection with partial labels based on gnn and pu learning. In *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 239–250.

[153] Lingzhe Zhang, Tong Jia, Mengxi Jia, Ying Li, Yong Yang, and Zhonghai Wu. 2024. Multivariate Log-based Anomaly Detection for Distributed Database. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4256–4267.

[154] Lingzhe Zhang, Tong Jia, Mengxi Jia, Hongyi Liu, Yong Yang, Zhonghai Wu, and Ying Li. 2024. Towards Close-To-Zero Runtime Collection Overhead: Raft-Based Anomaly Diagnosis on System Faults for Distributed Storage System. *IEEE Transactions on Services Computing* (2024).

[155] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Hongyi Liu, and Ying Li. 2025. ScalaLog: Scalable Log-Based Failure Diagnosis Using LLM. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.

[156] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Hongyi Liu, and Ying Li. 2025. XRAGLog: A Resource-Efficient and Context-Aware Log-Based Anomaly Detection Method Using Retrieval-Augmented Generation. In *AAAI 2025 Workshop on Preventing and Detecting LLM Misinformation (PDLM)*.

[157] Lingzhe Zhang, Tong Jia, Kangjin Wang, Mengxi Jia, Yong Yang, and Ying Li. 2024. Reducing Events to Augment Log-based Anomaly Detection Models: An Empirical Study. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 538–548.

[158] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Chiming Duan, Siyu Yu, Jinyang Gao, Bolin Ding, Zhonghai Wu, and Ying Li. 2025. ThinkFL: Self-Refining Failure Localization for Microservice Systems via Reinforcement Fine-Tuning. *arXiv preprint arXiv:2504.18776* (2025).

[159] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Xiaosong Huang, Chiming Duan, and Ying Li. 2025. AgentFM: Role-Aware Failure Management for Distributed Databases with LLM-Driven Multi-Agents. *arXiv preprint arXiv:2504.06614* (2025).

[160] Ling-Zhe Zhang, Xiang-Dong Huang, Yan-Kai Wang, Jia-Lin Qiao, Shao-Xu Song, and Jian-Min Wang. 2024. Time-tired compaction: An elastic compaction scheme for LSM-tree based time-series database. *Advanced Engineering Informatics* 59 (2024), 102224.

[161] Mingyang Zhang, Jianfei Chen, Jianyi Liu, Jingchu Wang, Rui Shi, and Hua Sheng. 2022. LogST: Log semi-supervised anomaly detection based on sentence-BERT. In *2022 7th International Conference on Signal and Image Processing (ICSIP)*. IEEE, 356–361.

[162] Shenglin Zhang, Yuhe Ji, Jiaqi Luan, Xiaohui Nie, Ziang Chen, Minghua Ma, Yongqian Sun, and Dan Pei. 2024. End-to-end automl for unsupervised log anomaly detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1680–1692.

[163] Shenglin Zhang, Sibo Xia, Wenzhao Fan, Binpeng Shi, Xiao Xiong, Zhenyu Zhong, Minghua Ma, Yongqian Sun, and Dan Pei. 2024. Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *arXiv preprint arXiv:2407.01710* (2024).

[164] Shenglin Zhang, Pengtian Zhu, Minghua Ma, Jiagang Wang, Yongqian Sun, Dongwen Li, Jingyu Wang, Qianying Guo, Xiaolei Hua, Lin Zhu, et al. 2024. Enhanced Fine-Tuning of Lightweight Domain-Specific Q&A Model Based on Large Language Models. In *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 61–66.

[165] Ting Zhang, Xin Huang, Wen Zhao, Shaohuang Bian, and Peng Du. 2023. LogPrompt: A Log-based Anomaly Detection Framework Using Prompts. In *2023 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.

[166] Tianyang Zhang, Zhuoxuan Jiang, Shengguang Bai, Tianrui Zhang, Lin Lin, Yang Liu, and Jiawei Ren. 2024. RAG4ITOps: A Supervised Fine-Tunable and Comprehensive RAG Framework for IT Operations and Maintenance. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 738–754.

[167] Wei Zhang, Xianfu Cheng, Yi Zhang, Jian Yang, Hongcheng Guo, Zhoujun Li, Xiaolin Yin, Xiangyuan Guan, Xu Shi, Liangfan Zheng, et al. 2024. ECLIPSE: Semantic Entropy-LCS for Cross-Lingual Industrial Log Parsing. *arXiv preprint arXiv:2405.13548* (2024).

[168] Wei Zhang, Hongcheng Guo, Jian Yang, Yi Zhang, Chaoran Yan, Zhoujin Tian, Hangyuan Ji, Zhoujun Li, Tongliang Li, Tieqiao Zheng, et al. 2024. mABC: multi-Agent Blockchain-Inspired Collaboration for root cause analysis in micro-services architecture. *arXiv preprint arXiv:2404.12135* (2024).

[169] Wanhao Zhang, Qianli Zhang, Enyu Yu, Yuxiang Ren, Yeqing Meng, Mingxi Qiu, and Jilong Wang. 2024. LogRAG: Semi-Supervised Log-based Anomaly Detection with Retrieval-Augmented Generation. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 1100–1102.

[170] Xuchao Zhang, Supriyo Ghosh, Chetan Bansal, Rujia Wang, Minghua Ma, Yu Kang, and Saravan Rajmohan. 2024. Automated root causing of cloud incidents using in-context learning with gpt-4. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 266–277.

[171] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).

[172] Aoxiao Zhong, Dengyao Mo, Guiyang Liu, Jinbu Liu, Qingda Lu, Qi Zhou, Jiesheng Wu, Quanzheng Li, and Qingsong Wen. 2024. Logparser-llm: Advancing efficient log parsing with large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4559–4570.

[173] Huazhen Zhong, Jibin Wang, Xuejian Wang, Xin Wang, Wenjie Xiao, Xuehai Tang, and Liangjun Zang. 2024. DBPrompt: A Database Anomaly Operation Detection and Analysis via Prompt Learning. In *International Conference on Intelligent Computing*. Springer, 357–368.

[174] Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. 2023. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems* 36 (2023), 43322–43355.

[175] Wubai Zhou, Liang Tang, Chunqiu Zeng, Tao Li, Larisa Shwartz, and Genady Ya Grabarnik. 2016. Resolution recommendation for event tickets in service management. *IEEE Transactions on Network and Service Management* 13, 4 (2016), 954–967.

[176] Xuanhe Zhou, Lianyuan Jin, Ji Sun, Xinyang Zhao, Xiang Yu, Jianhua Feng, Shifu Li, Tianqing Wang, Kun Li, and Luyang Liu. 2021. Dbmind: A self-driving platform in opengauss. *Proceedings of the VLDB Endowment* 14, 12 (2021), 2743–2746.

[177] Xuanhe Zhou, Guoliang Li, Zhaoyan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. 2023. D-bot: Database diagnosis system using large language models. *arXiv preprint arXiv:2312.01454* (2023).

[178] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2018. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering* 47, 2 (2018), 243–260.

[179] Xuanhe Zhou, Zhaoyan Sun, and Guoliang Li. 2024. Db-gpt: Large language model meets database. *Data Science and Engineering* 9, 1 (2024), 102–111.

[180] Xuanhe Zhou, Xinyang Zhao, and Guoliang Li. 2024. LLM-Enhanced Data Management. *arXiv preprint arXiv:2402.02643* (2024).